Stop Building Dumb Agents: Easy Database CRUD for Truly Intelligent Agents

Launch Intelligence
10.9K subscribers

Join | Subscribe | 46 | Share | Ask | Download

1,428 views Apr 29, 2025 Members first on March 6, 2025 #AI #APIs #Python
Join AI Dev Skool & Launch Your AI Startup Today! https://skool.com/ai-software-developers is the community for founders, builders, and AI innovators ready to take their projects to the next level.

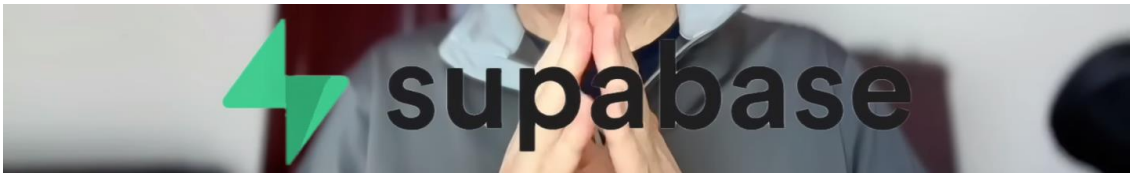
AI agents are powerful, but without structured data access, they lack reliability and scalability. In this tutorial, we'll integrate Supabase (PostgreSQL) with PydanticAI to create database-connected AI agents that can store, retrieve, and manage structured data efficiently.

🔻 What You'll Learn:
✅ Why databases are essential for AI agents
✅ How to use Supabase as a relational database for AI workflows
✅ Implementing CRUD (Create, Read, Update, Delete) operations
✅ Building context-aware database tools with PydanticAI
✅ Best practices for scalability, security, and performance

🔥 Why Database Agents Matter:
AI agents need persistent storage to handle:
✓ Data consistency & reliability for better decision-making
✓ Scalability to manage increasing data loads
✓ Security & compliance for enterprise and regulatory standards
✓ Efficient querying & retrieval using SQL
✓ Long-term storage & analytics for AI model improvement

📌 Topics Covered:
🔻 Setting up Supabase & connecting it to PydanticAI
🔻 Creating database tools for querying, inserting, updating, and deleting data
🔻 Handling error management and performance optimization
🔻 Real-world demo: Building a CRUD-enabled AI agent

🎓 Full-Stack Agentic AI Masterclass Series:
- Part 1: Introduction
- Part 2: The AI framework landscape in 2025
- Part 3: Building effective agents - principles & practice
- Part 4: Prompt Engineering for effective agents
- Part 5: • Your AI Agents Are Useless Without This: P...
- Part 6: Building UIs for your agents
- Part 7: • Stop Building Dumb Agents: Easy Database C...
- Part 7.5: • AI That Writes SQL: Automate Queries with ...
- Part 8: RAG
- Part 9: Integrating PydanticAI with N8N
- Part 10: Containers and deployment
- Part 11: E2E Final Project - customer support agent with PydanticAI



WHY DB Agents?

Business enablers

1. **Data Consistency & Reliability** – Storing structured data in a database ensures that AI agents have a single source of truth, reducing errors and inconsistencies in decision-making.

2. **Scalability** – Databases allow AI agents to handle increasing volumes of structured data efficiently, supporting business growth without performance bottlenecks.

3. **Compliance & Security** – Many industries require data retention, auditability, and strict access controls. Databases provide encryption, authentication, and logging to meet re ory standards.

WHY DB Agents?

Technical reasons

1. **Efficient Querying & Retrieval** – Unlike storing data in files or volatile memory, databases allow AI agents to perform complex queries, aggregations, and searches efficiently with SQL.

2. **Integration with External Systems** – AI agents often need to interact with CRM, ERP, and analytics tools. Databases provide structured, standardized data access, making integrations seamless.

3. **Long-Term Storage & Analysis** – AI agents improve when they have access to hist data for trend analysis, training, and fine-tuning—something databases handle far be n in-memory storage.

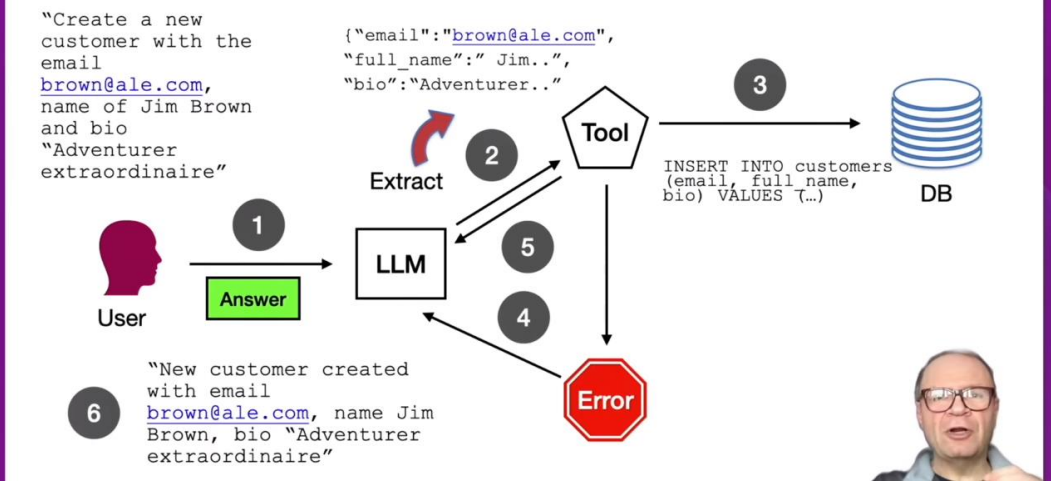
What We're Building

CRUD Agent with PydanticAI and Supabase

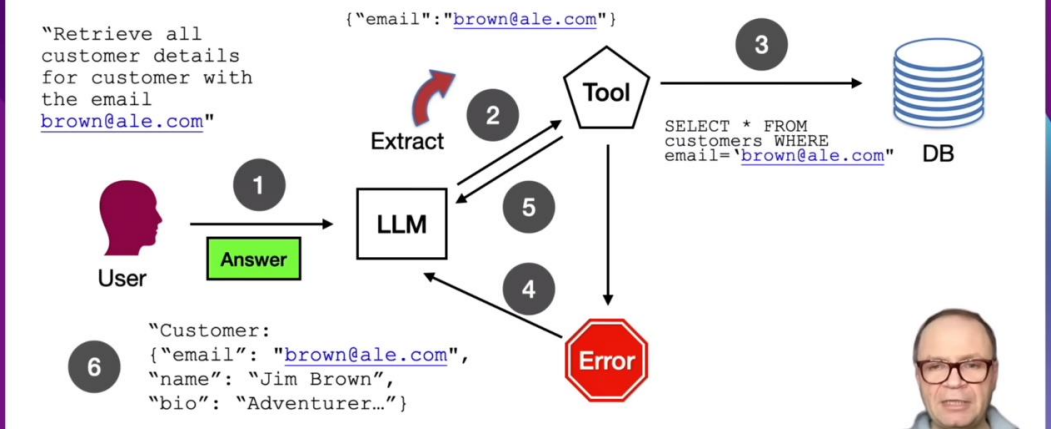
CRUD Operations

1. **Create a record** – a databse insert tool to create a new customer
2. **Retrieve a record** – a databse query tool to search for and return a customer
3. **Update a record** – a database tool to find a customer by email and update the record
4. **Delete a record** – a database tool to find a customer by email and de the record

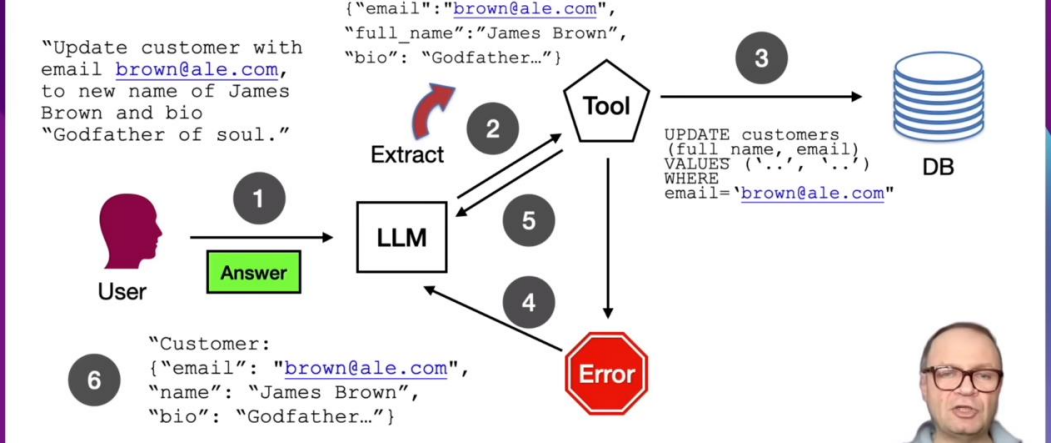
Agent Design - Create



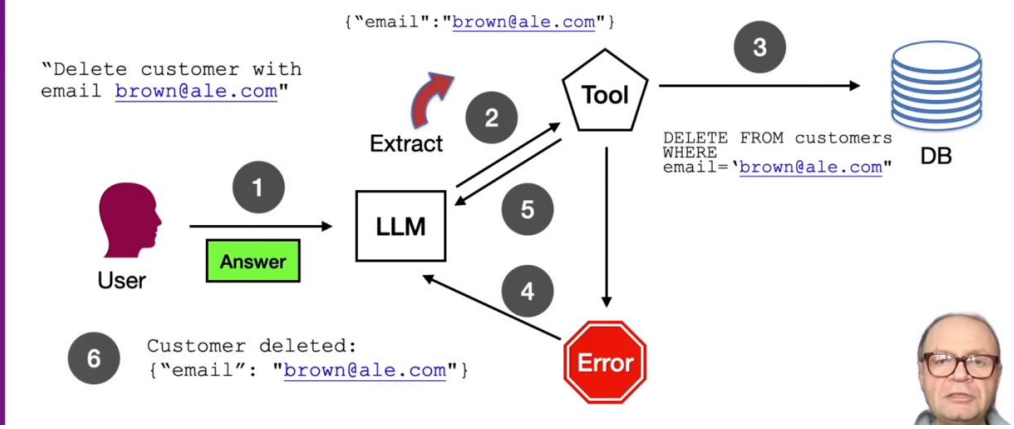
Agent Design - Retrieve



Agent Design - Update



Agent Design - Delete

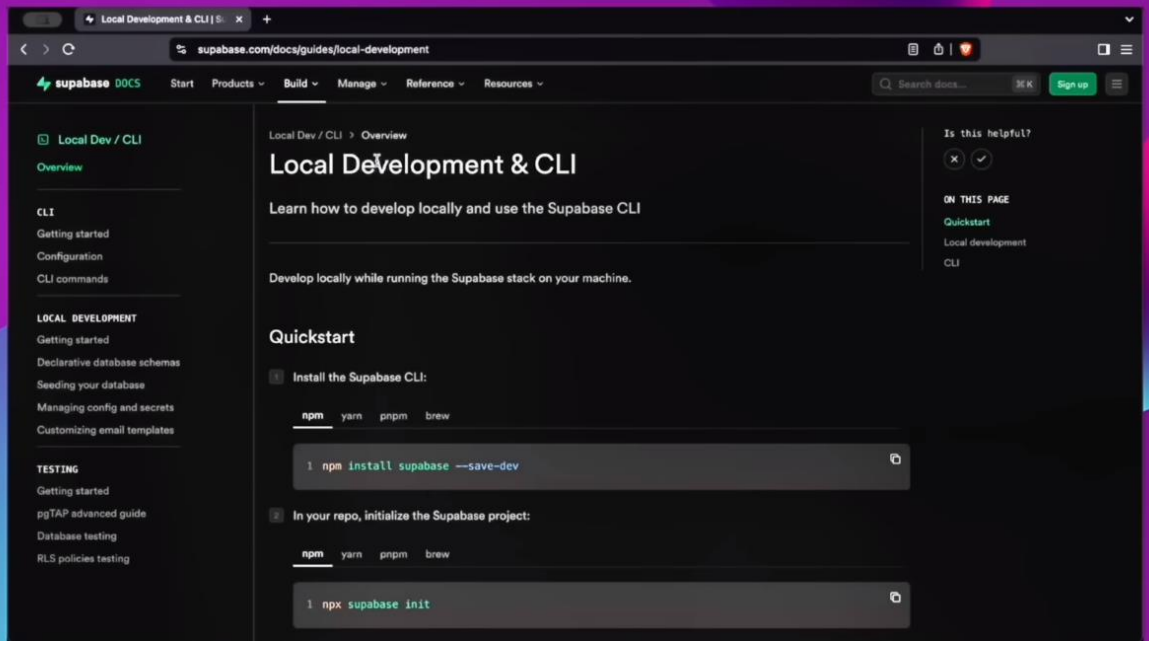


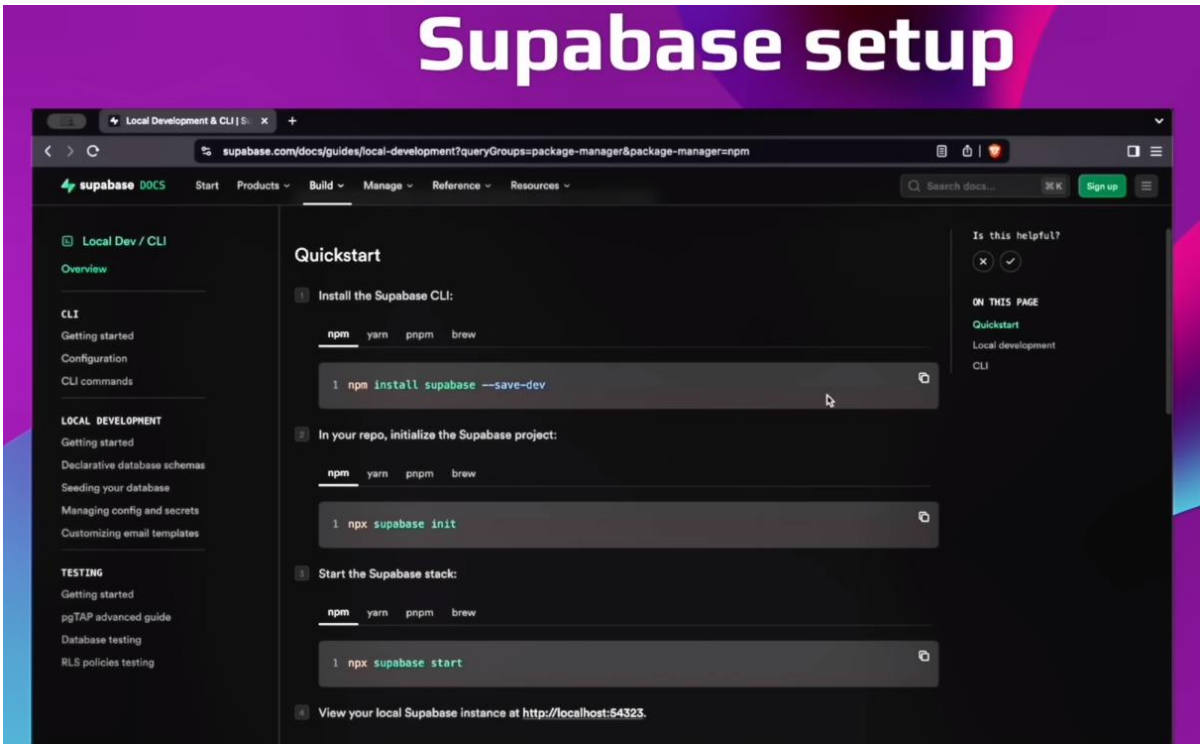
Coding Tutorial

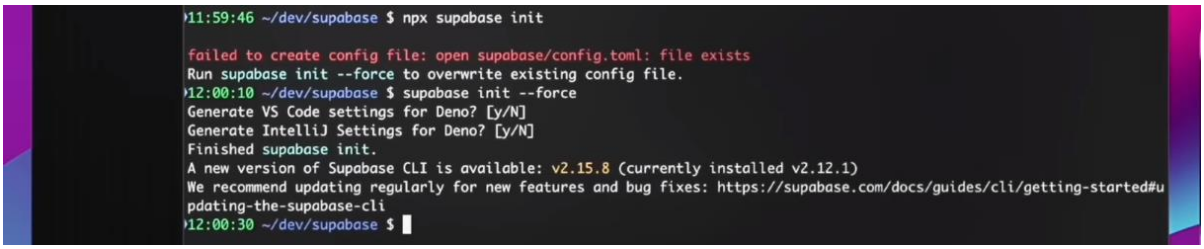
Database Agents with PydanticAI

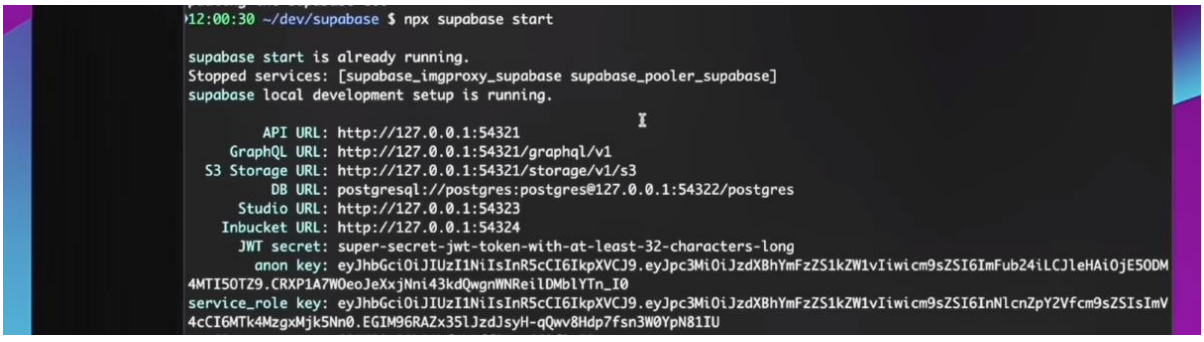
1. Download and install **Docker Desktop**
2. Download and install **Supabase**
3. Create and configure the **customers** table – Create table, Create unique column index (email), Disable RLS
4. Create **basic agent** – write a basic agent with Supabase client access and DB seed function
5. Create **CRUD tools** – create, retrieve, update, delete tools for Supab
6. **Test agent** – perform basic CRUD tests

Supabase setup

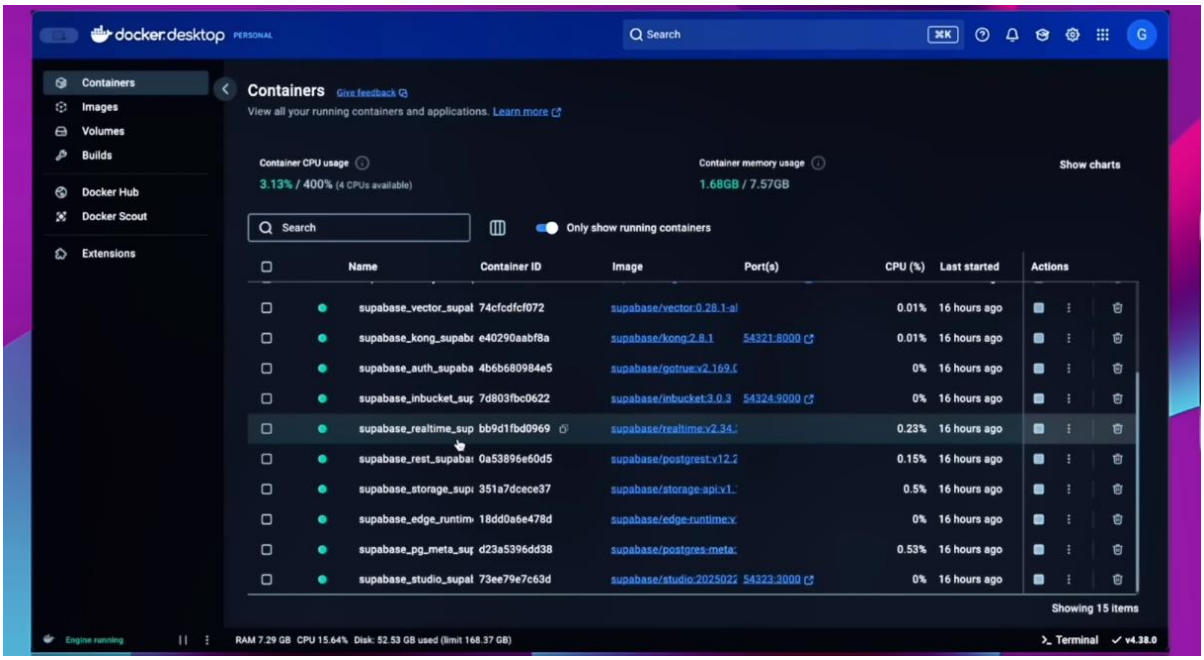


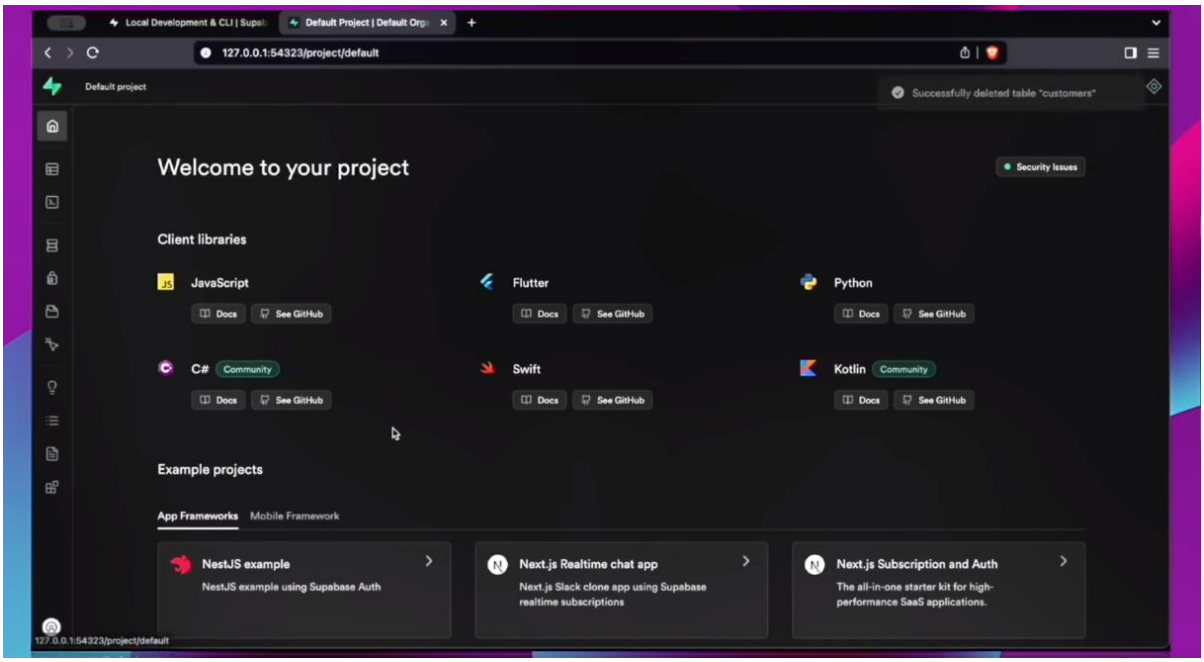




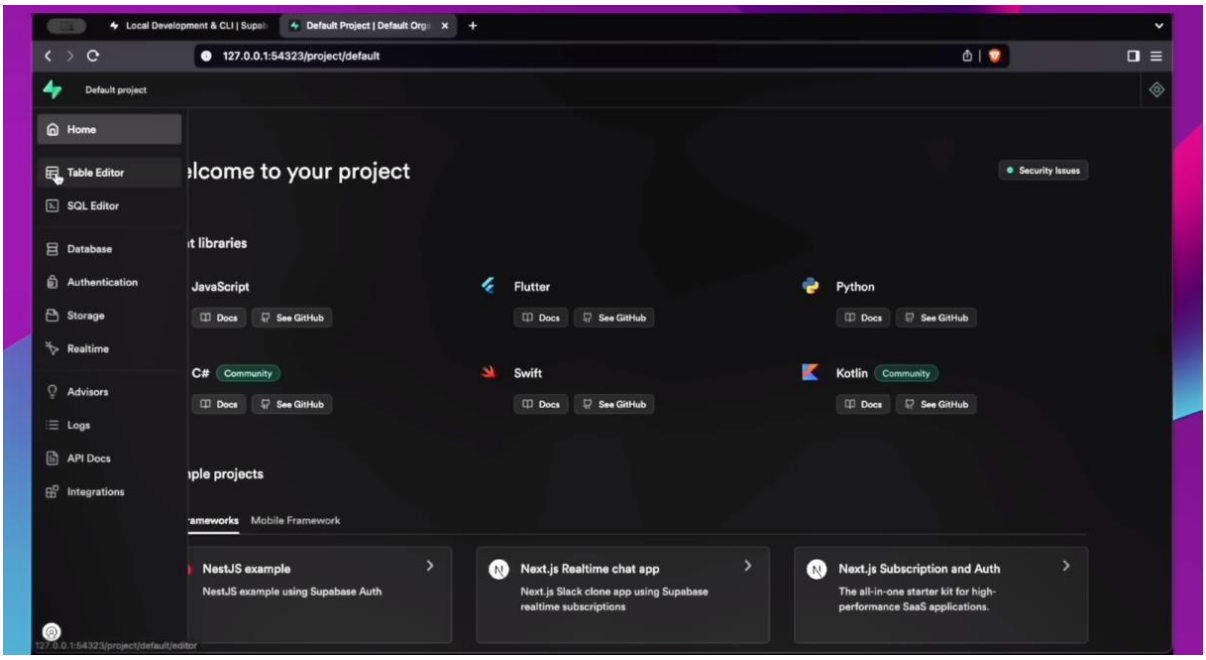


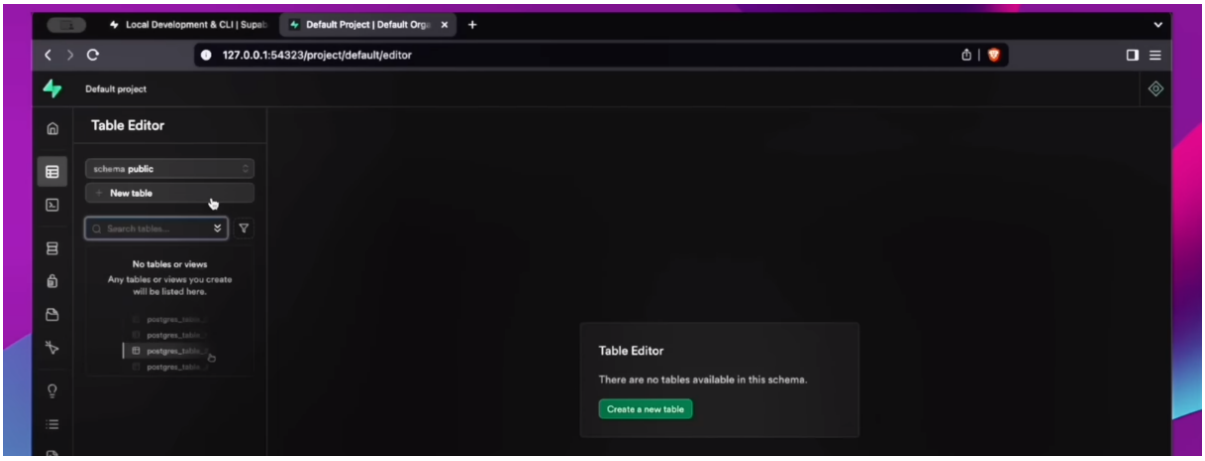
Copy the URLs into some new location

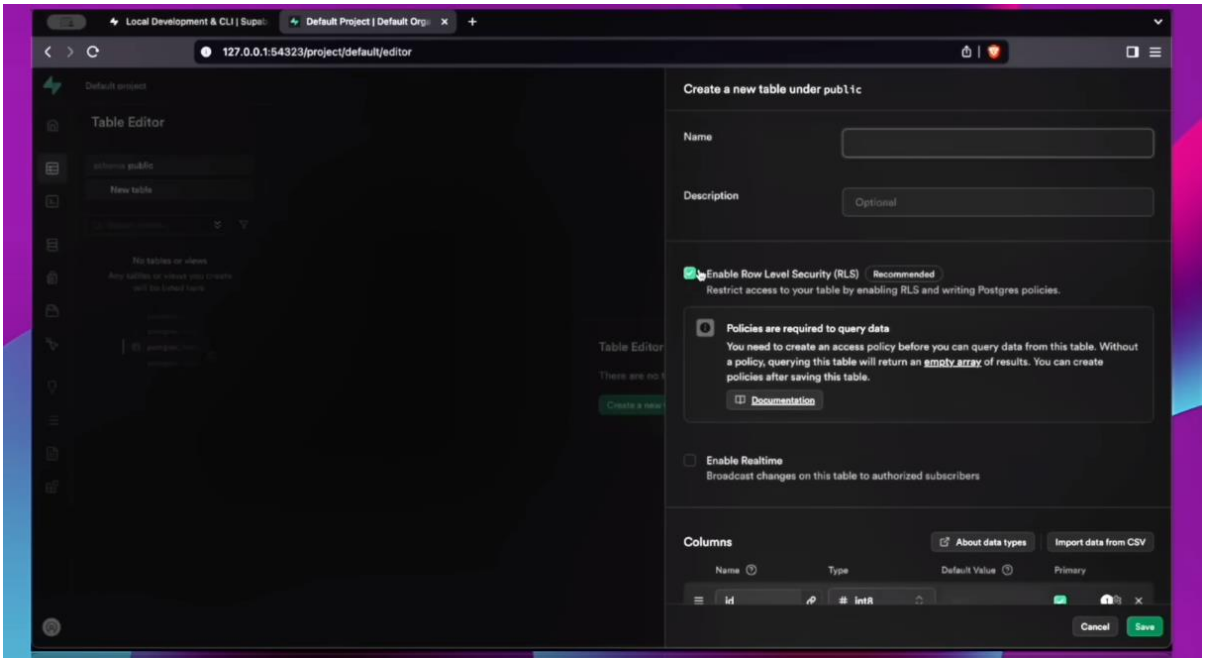


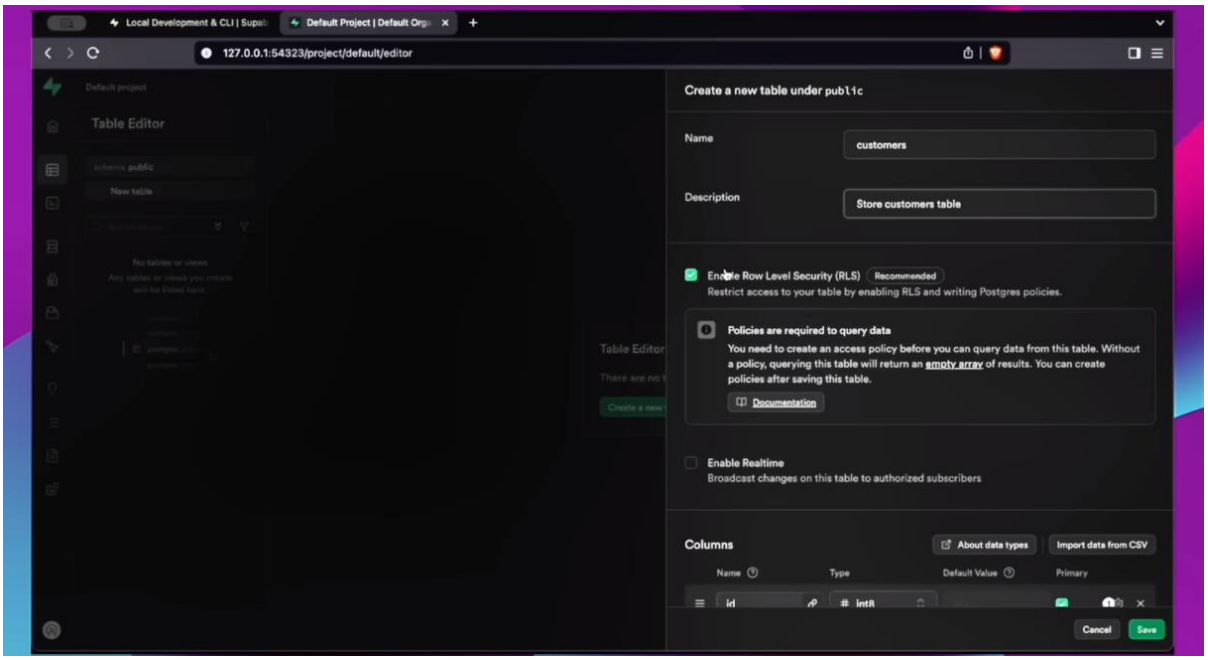


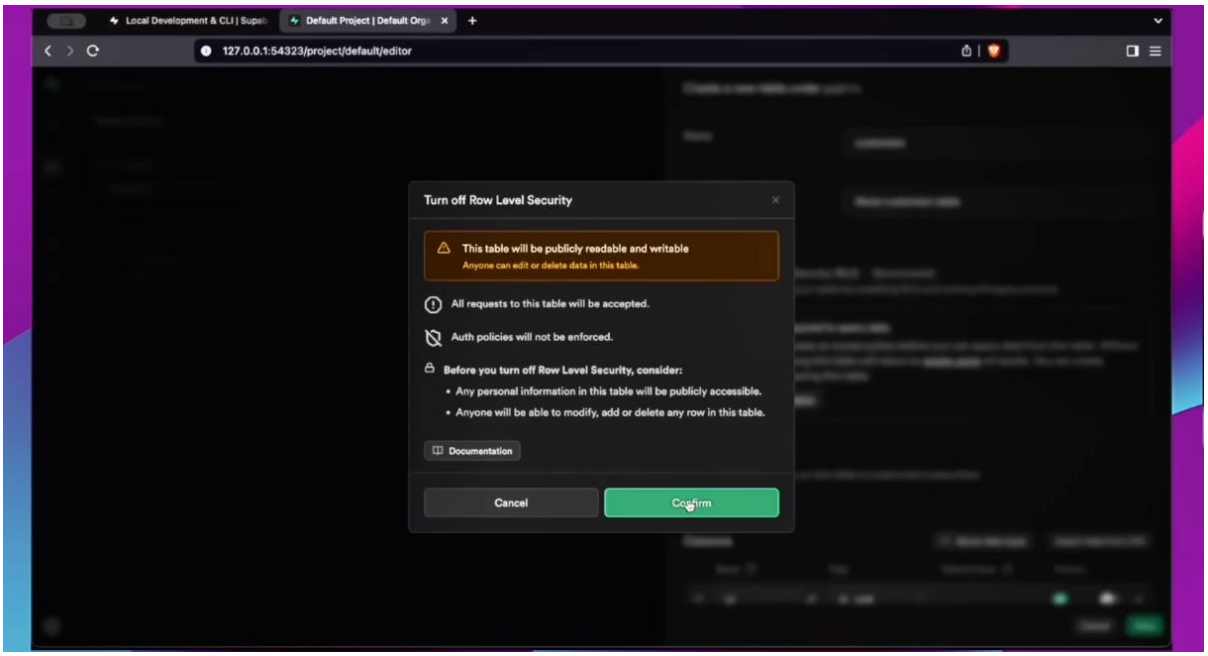
Our Superbase installation is successful and working as seen in the Studio URL above

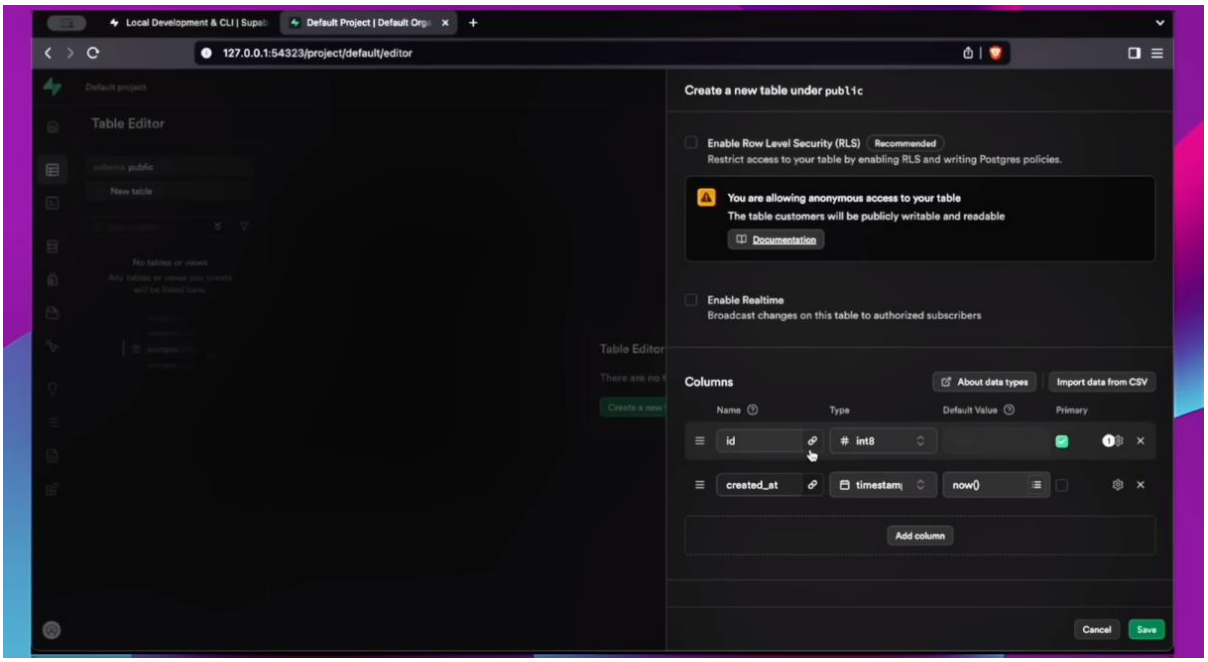

Local Development & CLI | Supab
Default Project | Default Orga
127.0.0.1:54323/project/default
Default project
Home
Table Editor
SQL Editor
Database
Authentication
Storage
Realtime
Advisors
Logs
API Docs
Integrations
lcome to your project
Security Issues
t libraries
JavaScript
Docs
See GitHub
Flutter
Python
C# Community
Swift
Kotlin Community
ple projects
rameworks
Mobile Framework
NestJS example
NestJS example using Supabase Auth
Next.js Realtime chat app
Next.js Slack clone app using Supabase realtime subscriptions
Next.js Subscription and Auth
The all-in-one starter kit for high-performance SaaS applications.
127.0.0.1:54323/project/default/editor


Local Development & CLI | Supab
Default Project | Default Orga
127.0.0.1:54323/project/default/editor
Default project
Table Editor
schema public
New table
Search tables...
No tables or views
Any tables or views you create will be listed here.
Table Editor
There are no tables available in this schema.
Create a new table

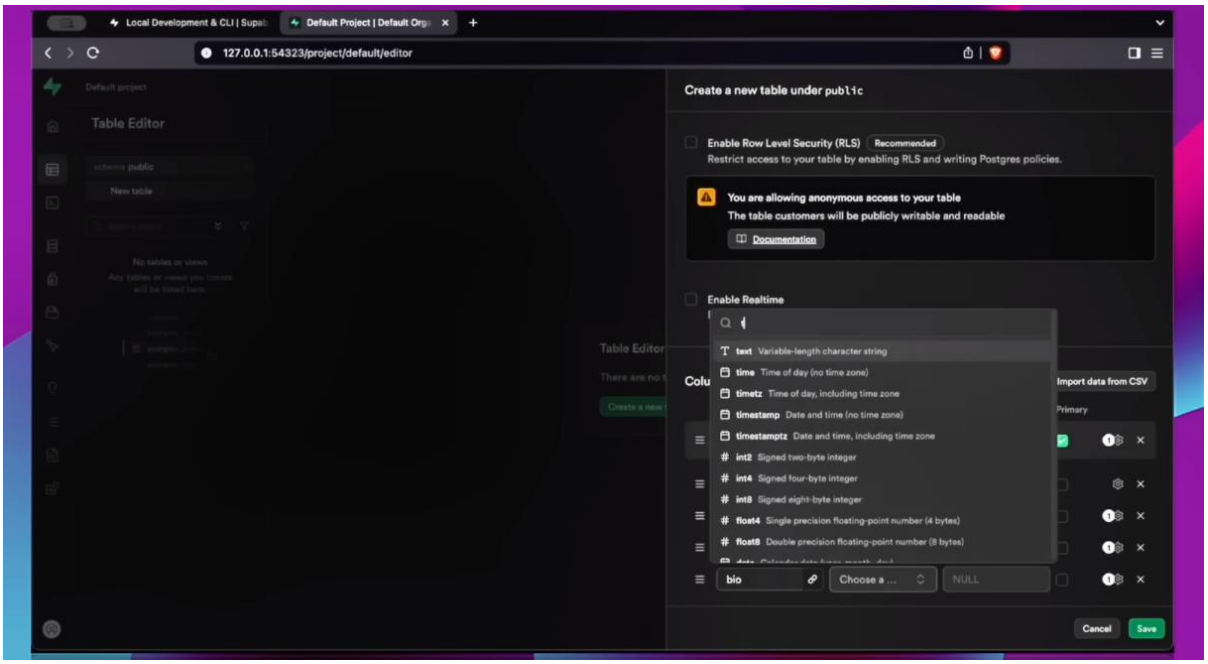

Local Development & CLI | Supab
Default Project | Default Orga
127.0.0.1:54323/project/default/editor
Default project
Table Editor
Create a new table under public
Name
Description
Optional
Enable Row Level Security (RLS)
Recommended
Restrict access to your table by enabling RLS and writing Postgres policies.
Policies are required to query data
You need to create an access policy before you can query data from this table. Without a policy, querying this table will return an empty array of results. You can create policies after saving this table.
Documentation
Enable Realtime
Broadcast changes on this table to authorized subscribers
Columns
About data types
Import data from CSV
Name
Type
Default Value
Primary
id
int8
Cancel
Save

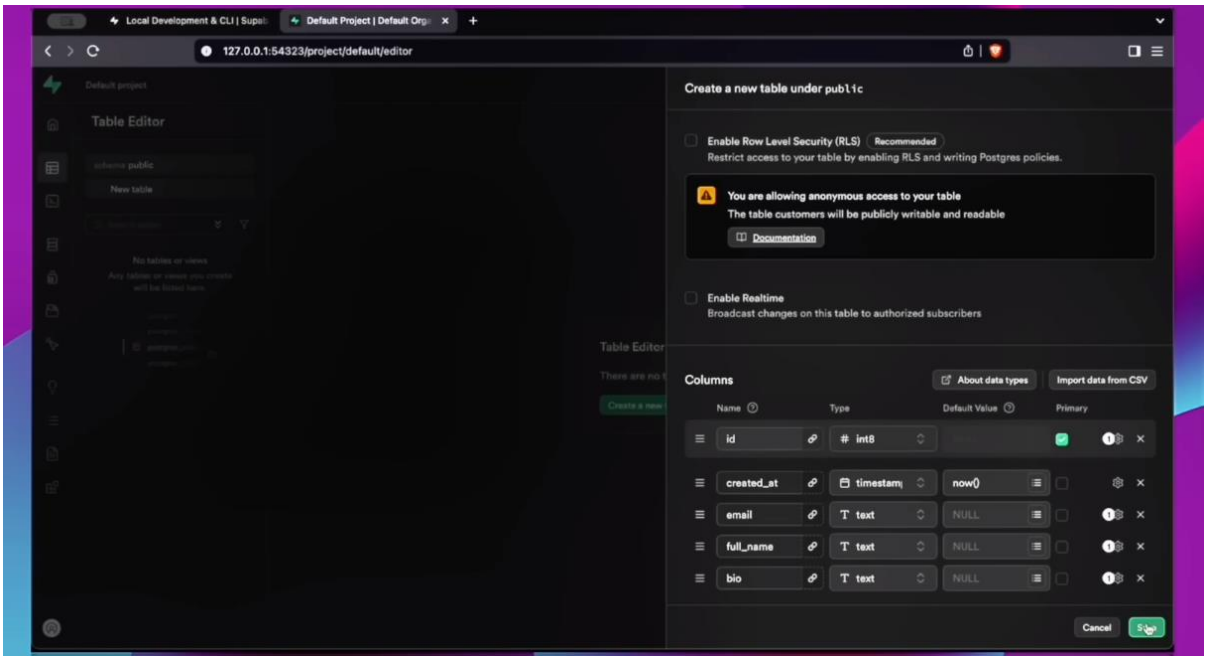

Local Development & CLI | Supa
Default Project | Default Org
127.0.0.1:54323/project/default/editor
Table Editor
Create a new table under public
Name
customers
Description
Store customers table
Enable Row Level Security (RLS)
Recommended
Restrict access to your table by enabling RLS and writing Postgres policies.
Policies are required to query data
You need to create an access policy before you can query data from this table. Without a policy, querying this table will return an empty array of results. You can create policies after saving this table.
Documentation
Enable Realtime
Broadcast changes on this table to authorized subscribers
Columns
About data types
Import data from CSV
Name
Type
Default Value
Primary
id
int8
Cancel
Save

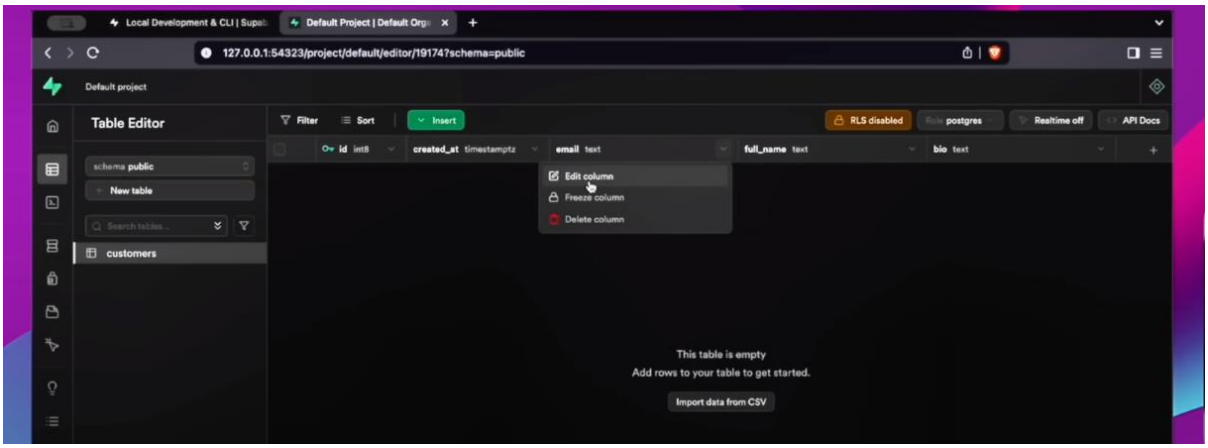

Local Development & CLI | Supa
Default Project | Default Org
127.0.0.1:54323/project/default/editor
Turn off Row Level Security
This table will be publicly readable and writable
Anyone can edit or delete data in this table.
All requests to this table will be accepted.
Auth policies will not be enforced.
Before you turn off Row Level Security, consider:
Any personal information in this table will be publicly accessible.
Anyone will be able to modify, add or delete any row in this table.
Documentation
Cancel
Confirm



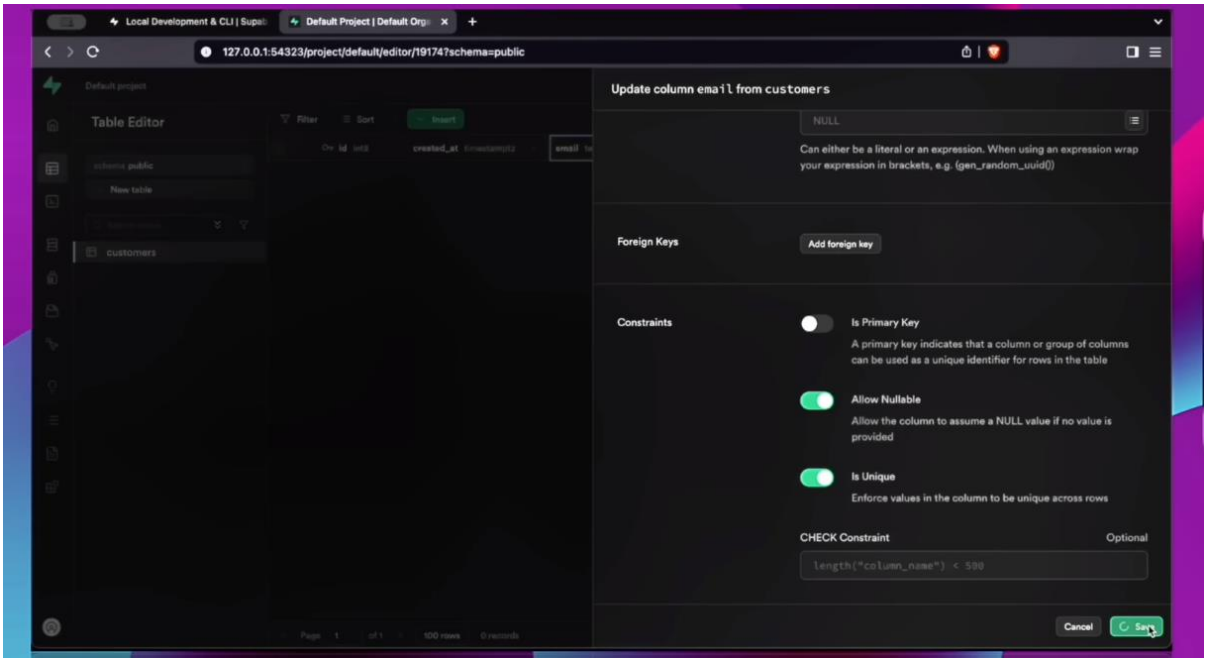
We need to add 3 new columns as below

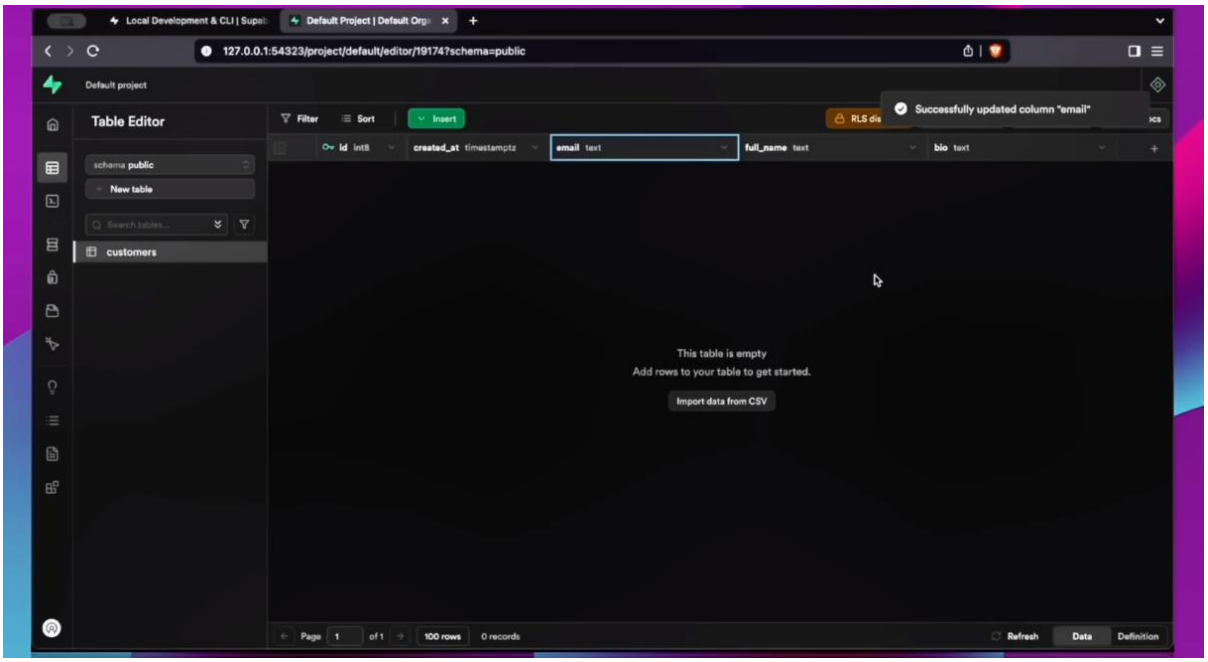




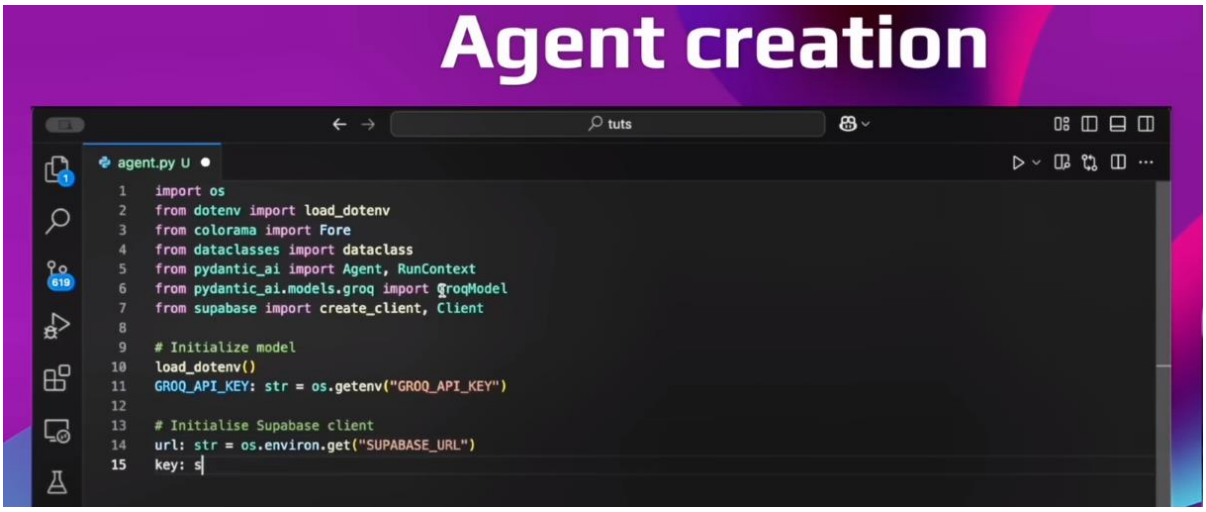


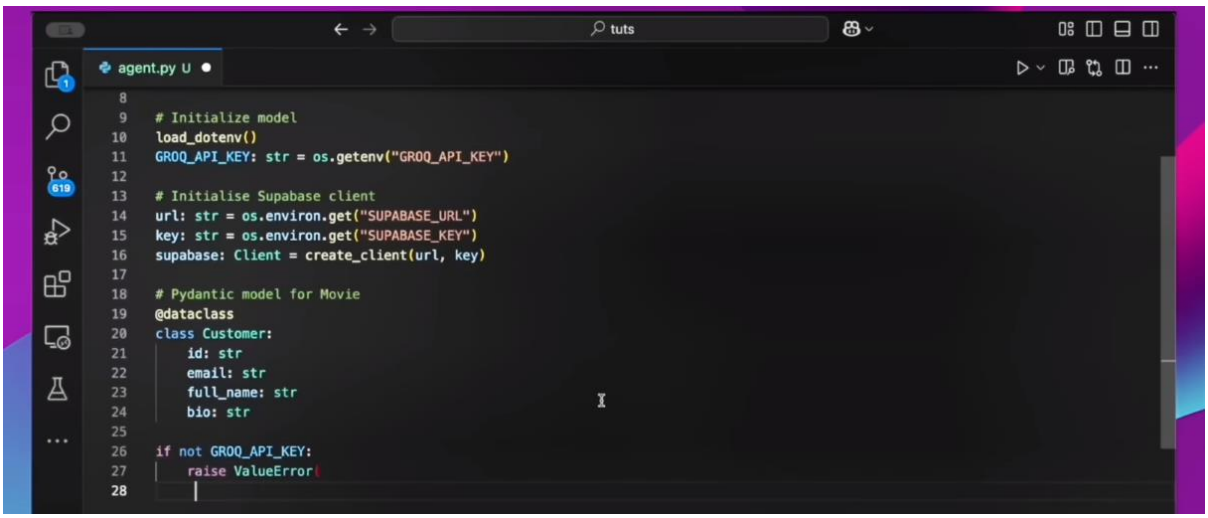
We need to make the email column unique in our database table

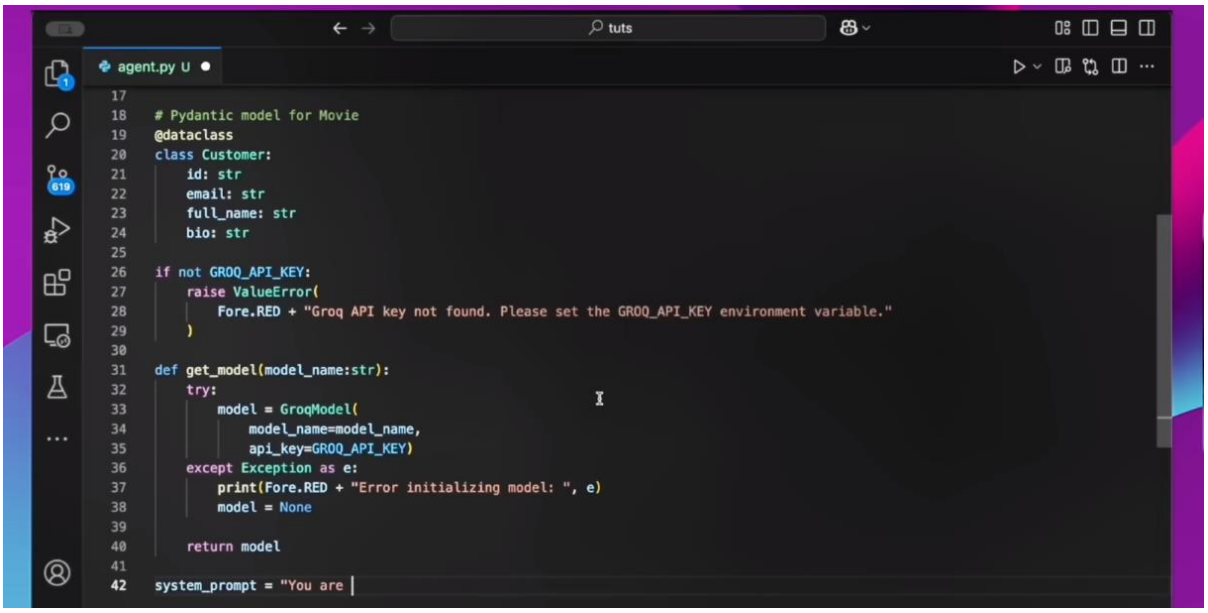


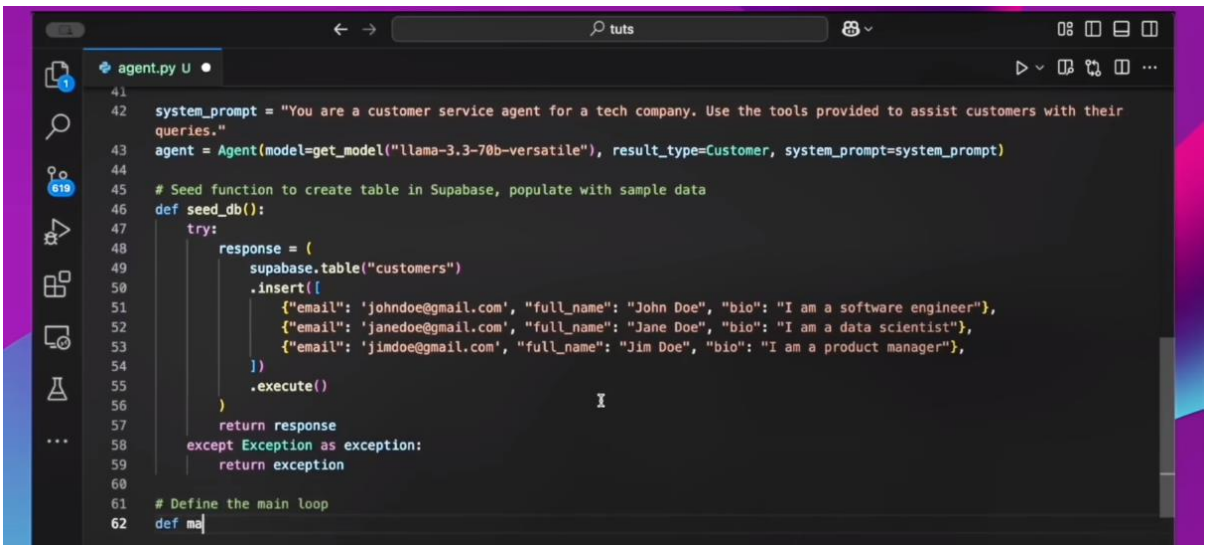


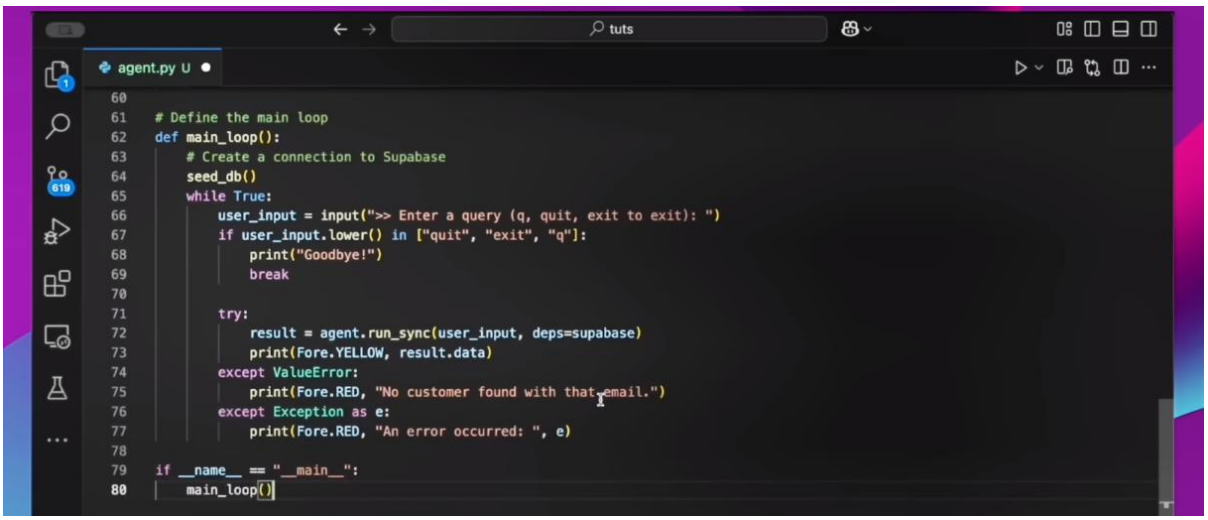
We are now ready to use our Customers table in Superbase

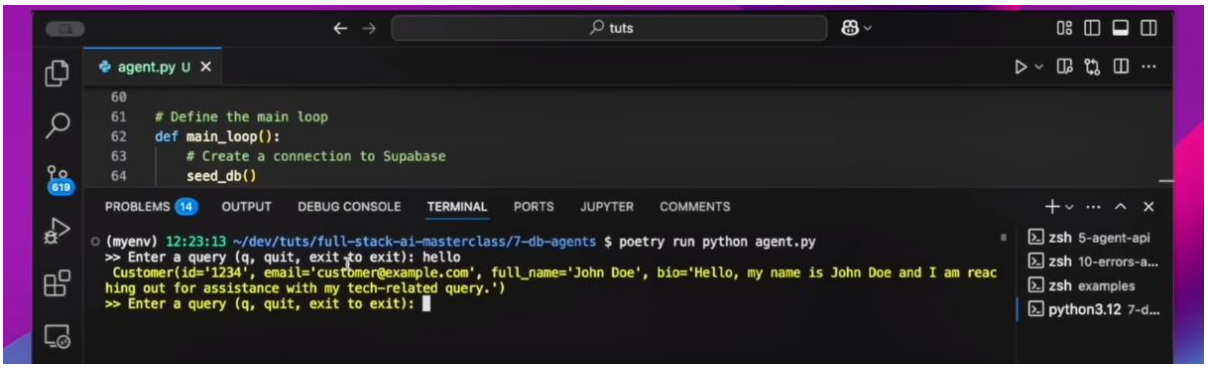


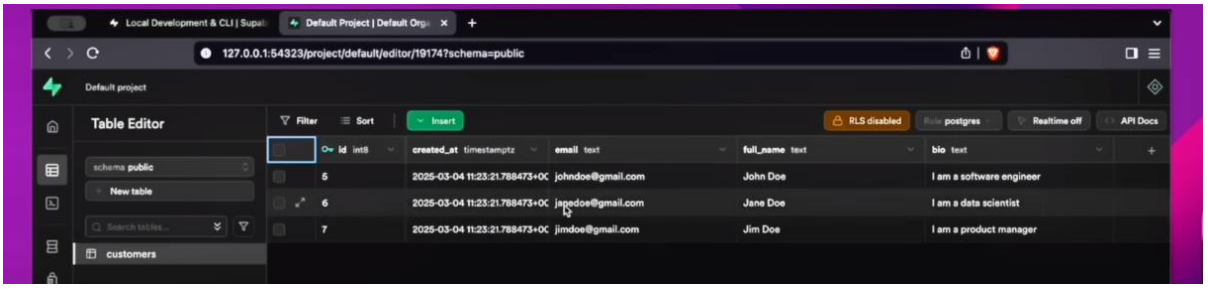


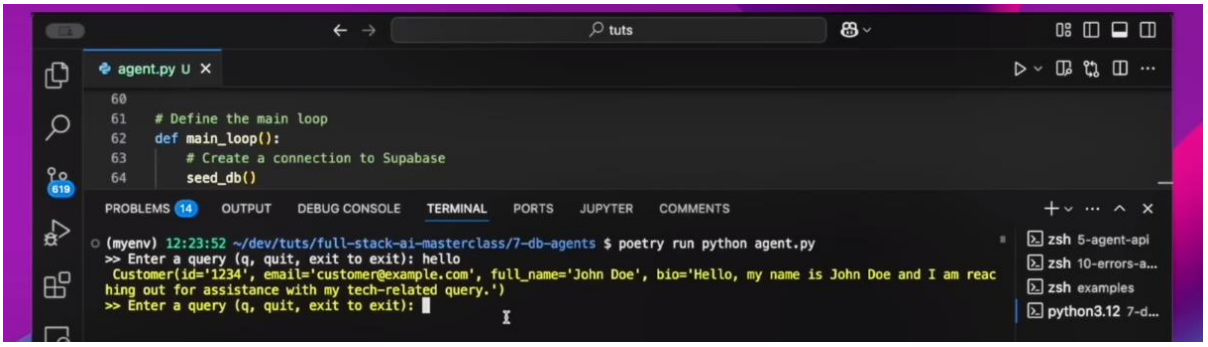

tuts
agent.py U
Pydantic model for Movie
@dataclass
class Customer:
id: str
email: str
full_name: str
bio: str
if not GROQ_API_KEY:
raise ValueError(
Fore.RED + "Groq API key not found. Please set the GROQ_API_KEY environment variable."
)
def get_model(model_name:str):
try:
model = GroqModel(
model_name=model_name,
api_key=GROQ_API_KEY)
except Exception as e:
print(Fore.RED + "Error initializing model: ", e)
model = None
return model
system_prompt = "You are

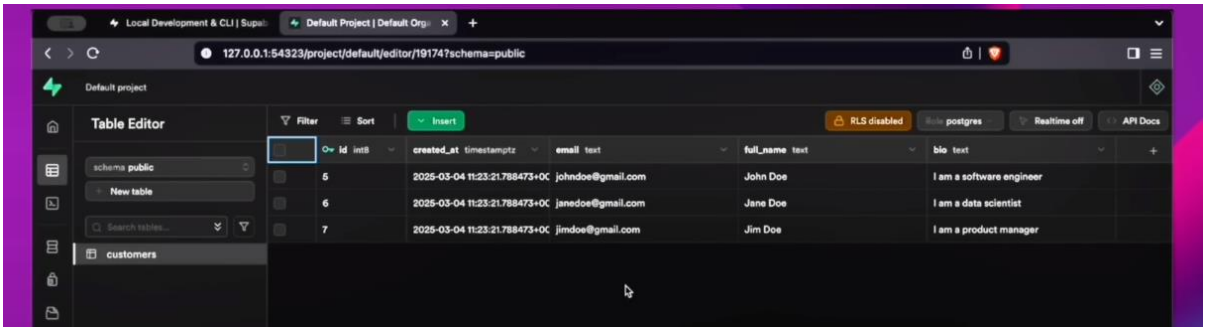

tuts
agent.py U
system_prompt = "You are a customer service agent for a tech company. Use the tools provided to assist customers with their queries."
agent = Agent(model=get_model("llama-3.3-70b-versatile"), result_type=Customer, system_prompt=system_prompt)
# Seed function to create table in Supabase, populate with sample data
def seed_db():
try:
response = (
supabase.table("customers")
.insert([
{"email": 'johndoe@gmail.com', "full_name": "John Doe", "bio": "I am a software engineer"},
{"email": 'janedoe@gmail.com', "full_name": "Jane Doe", "bio": "I am a data scientist"},
{"email": 'jimdoe@gmail.com', "full_name": "Jim Doe", "bio": "I am a product manager"},
])
.execute()
)
return response
except Exception as exception:
return exception
Define the main loop
def ma


tuts
agent.py U
# Define the main loop
def main_loop():
# Create a connection to Supabase
seed_db()
while True:
user_input = input(">> Enter a query (q, quit, exit to exit): ")
if user_input.lower() in ["quit", "exit", "q"]:
print("Goodbye!")
break
try:
result = agent.run_sync(user_input, deps=supabase)
print(Fore.YELLOW, result.data)
except ValueError:
print(Fore.RED, "No customer found with that email.")
except Exception as e:
print(Fore.RED, "An error occurred: ", e)
if __name__ == "__main__":
main_loop()

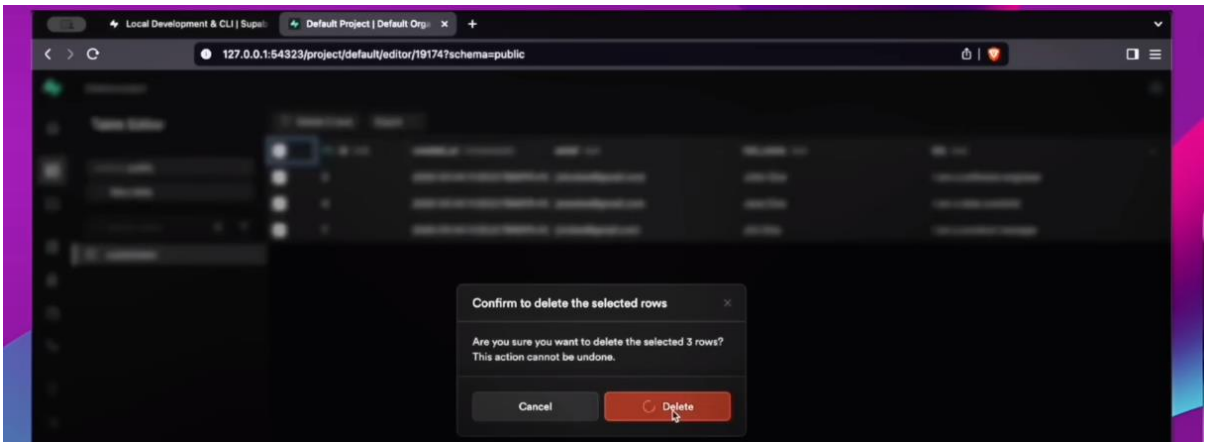


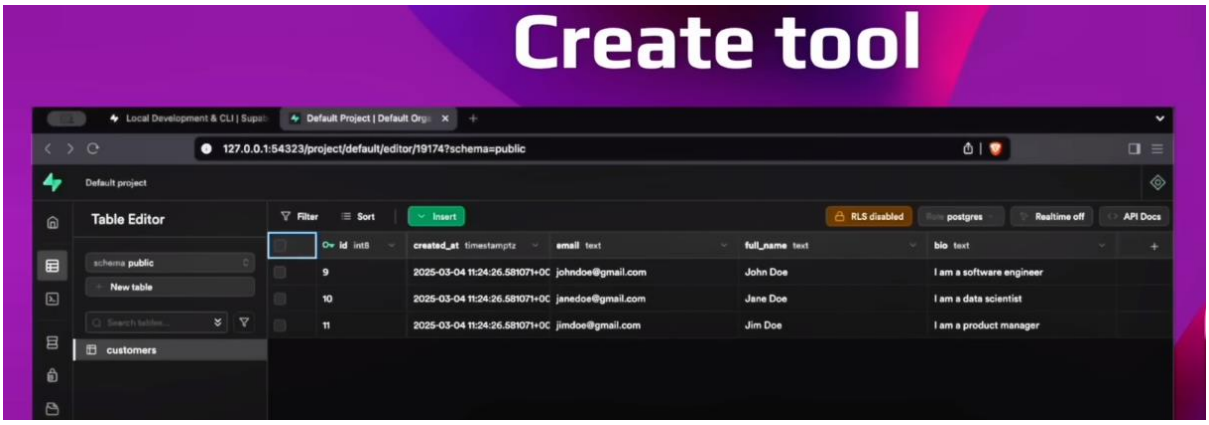




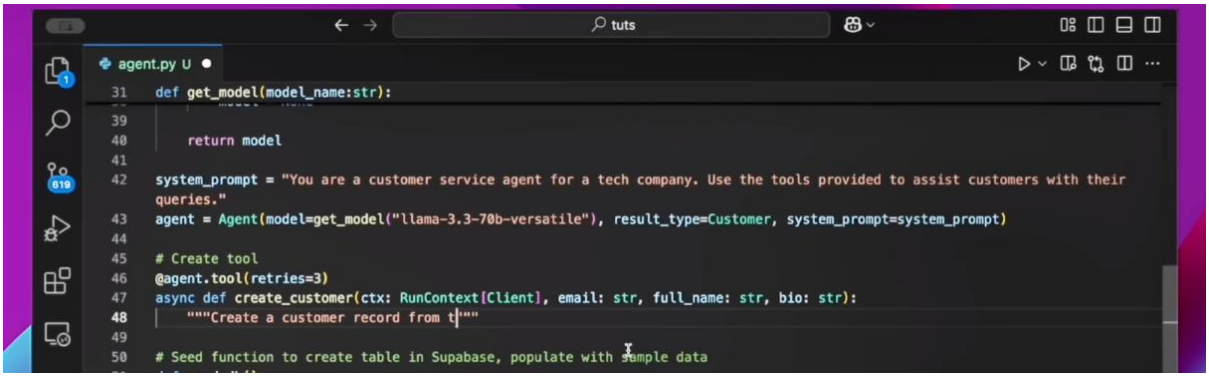


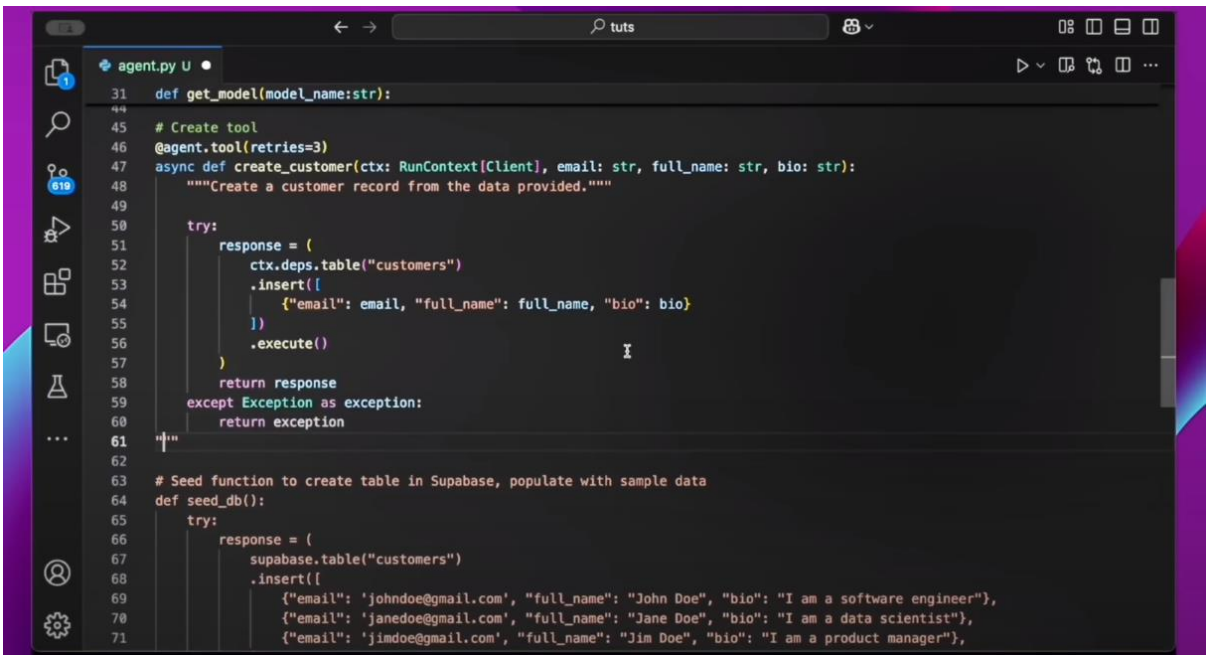
After restarting and seeding the DB again, we still have 3 rows because we made the email column unique

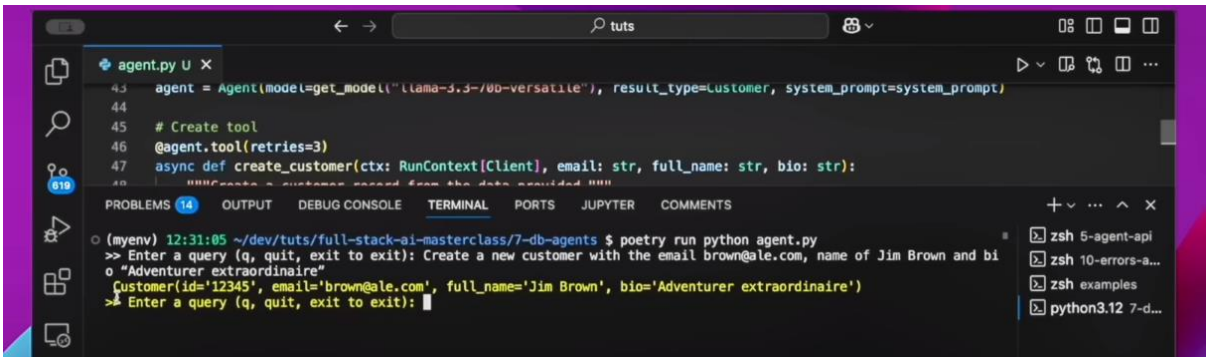


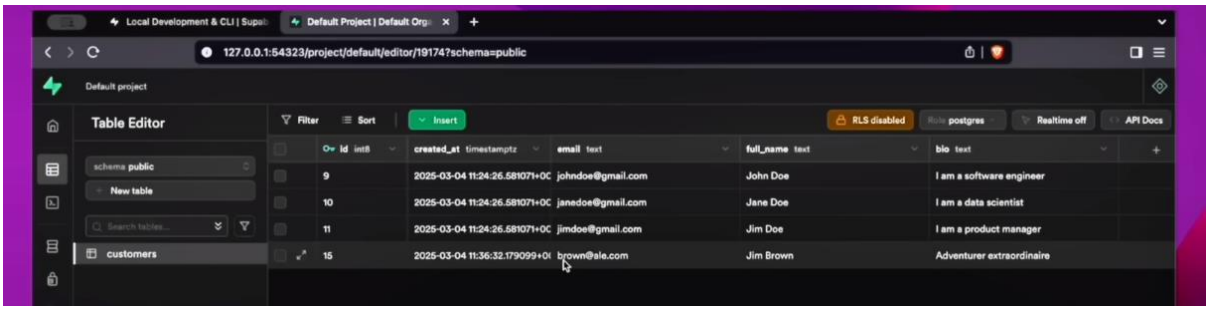


Empty the table and rerun the seeding again. We can now go ahead and start creating our agent tools as below.

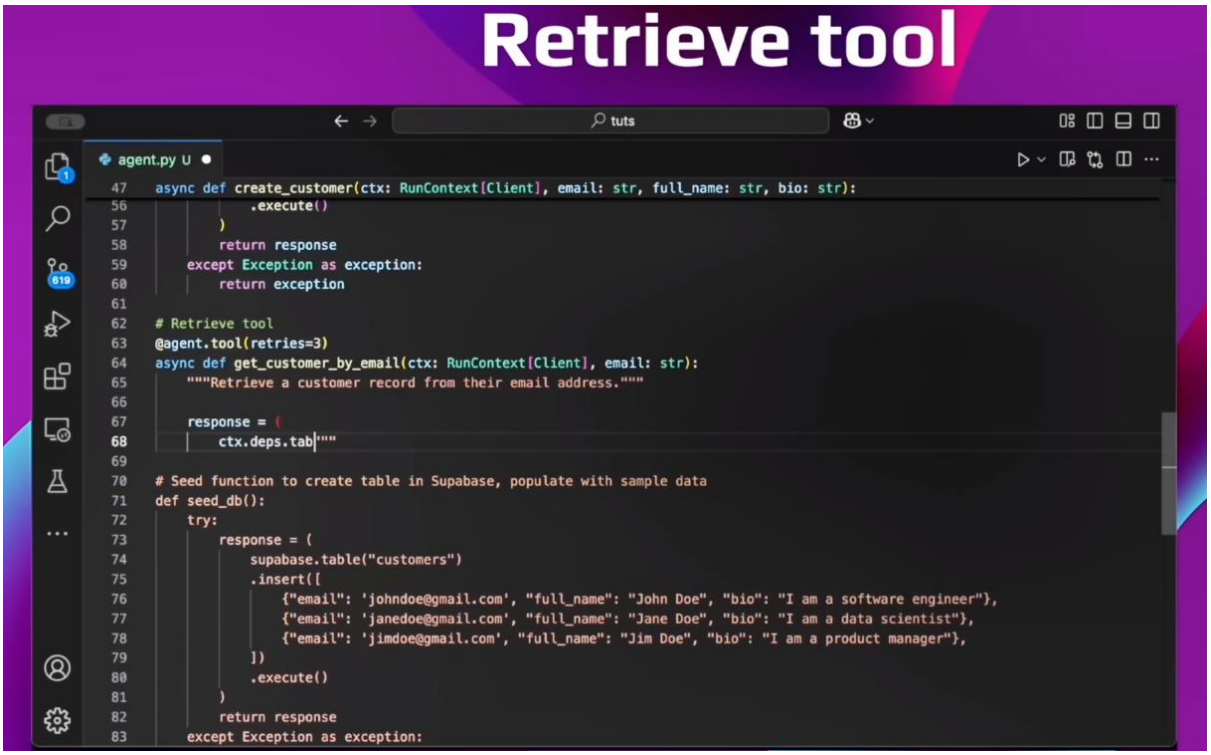


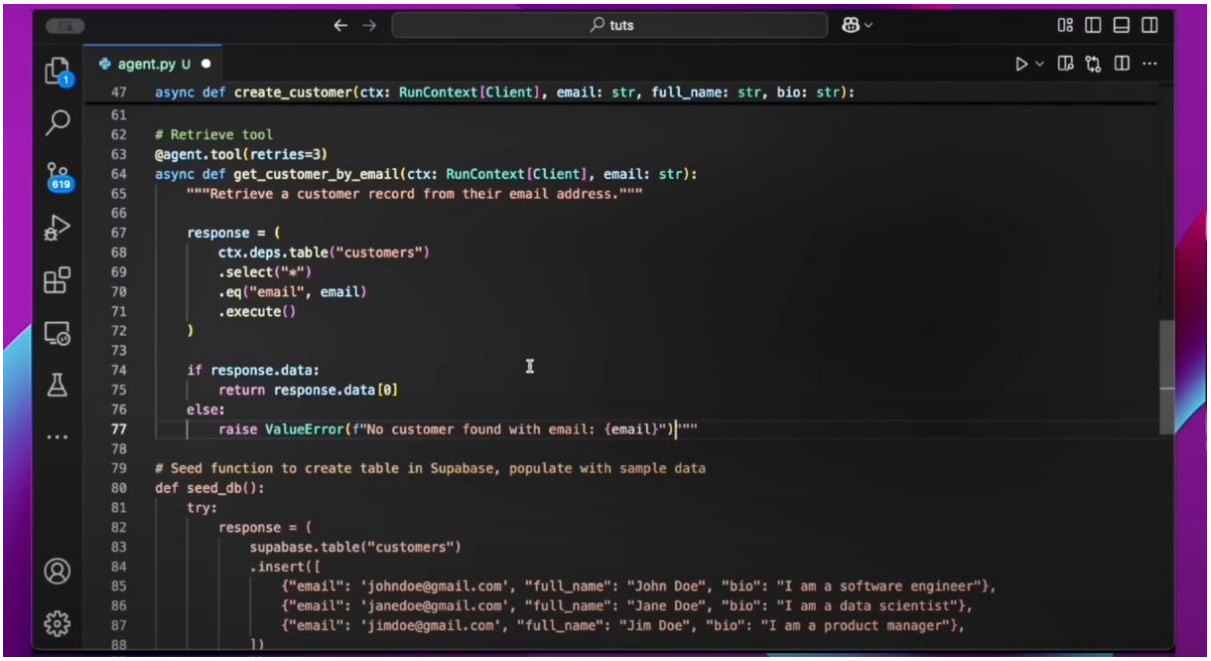


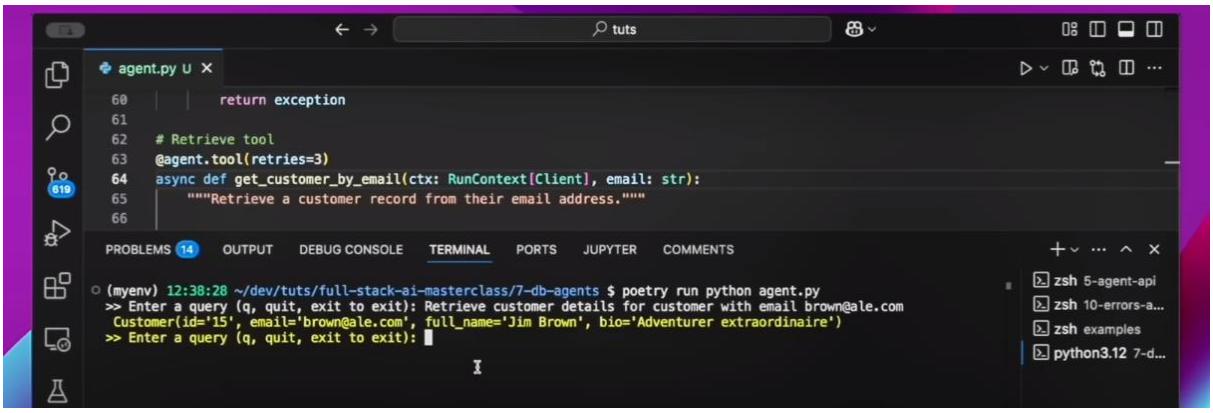


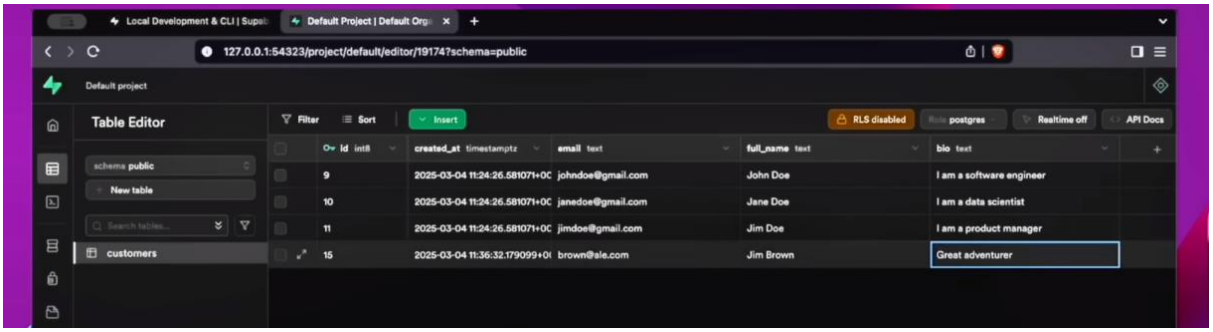


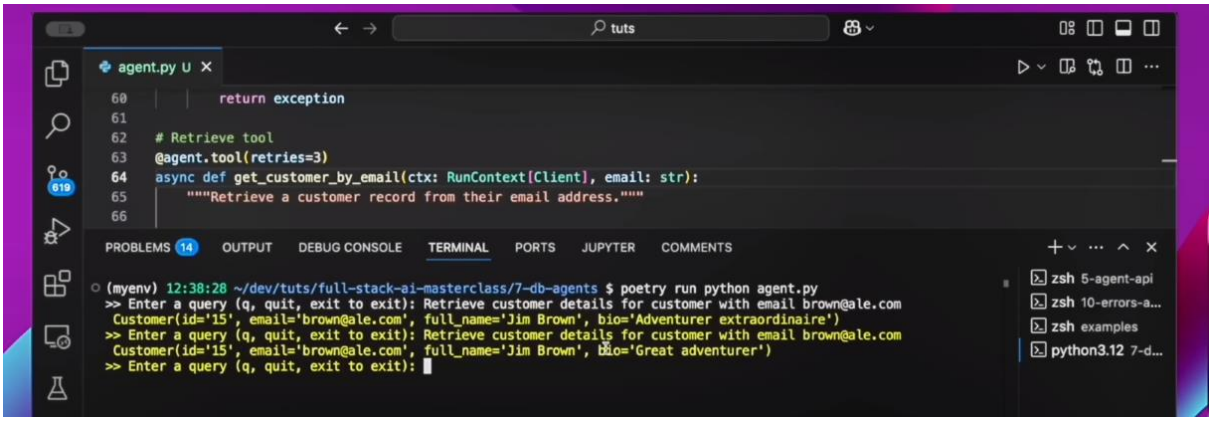
We have the new data inserted using the Create tool





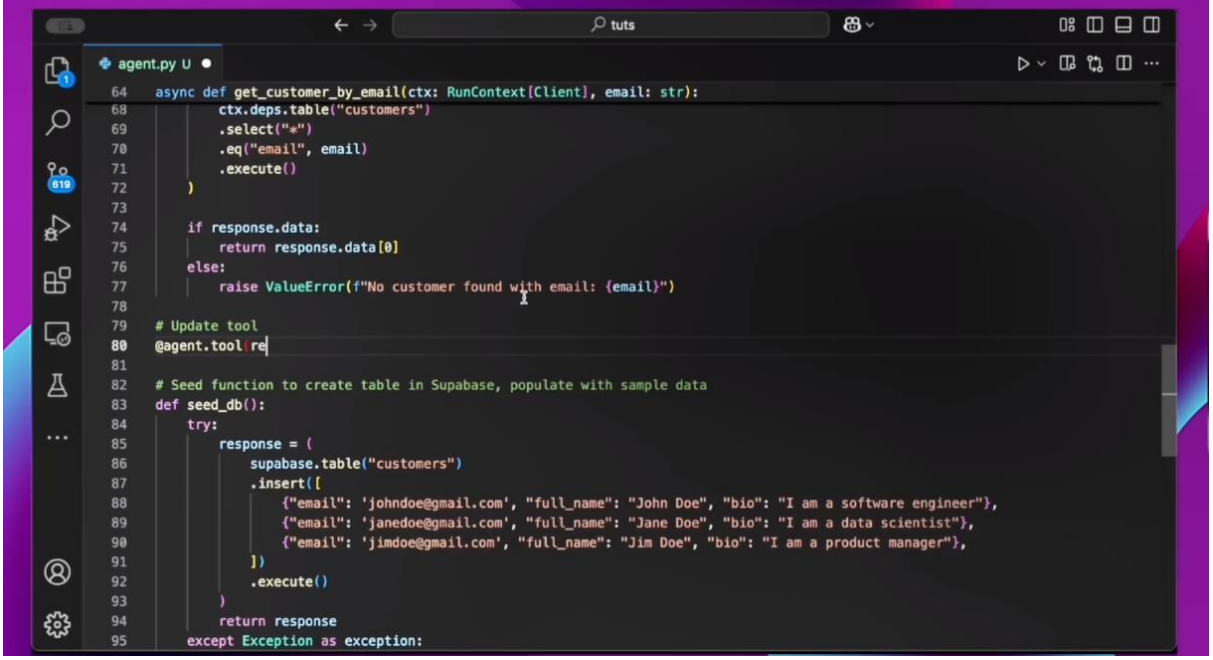




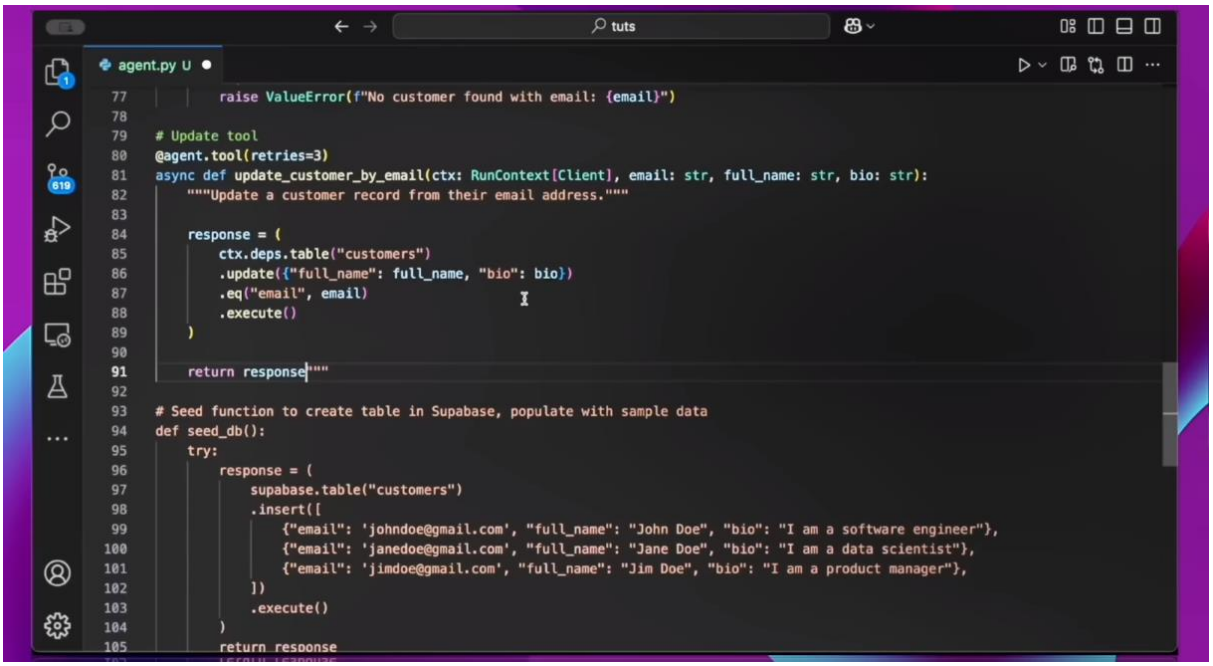


This proves that the agent is making a real-time access call to the DB and not using cached data

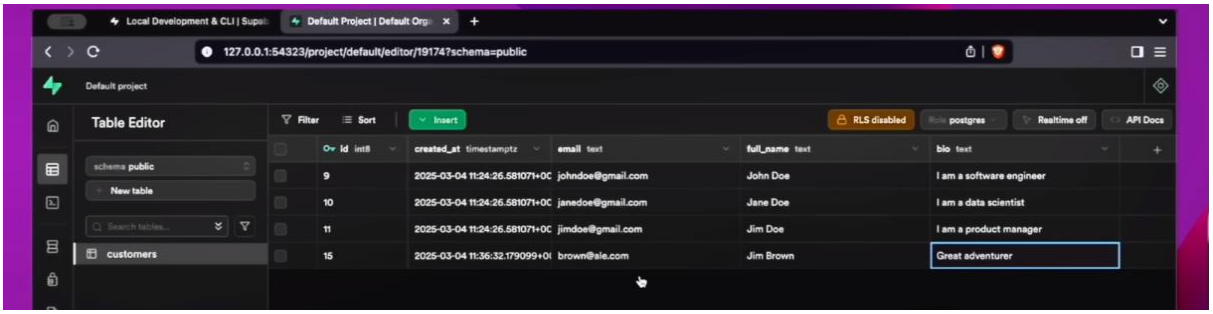
Update tool

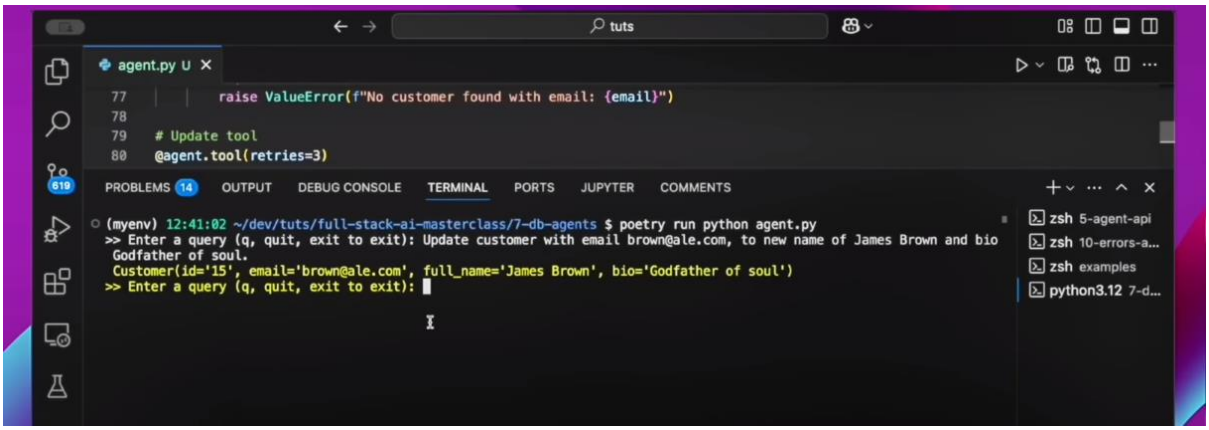


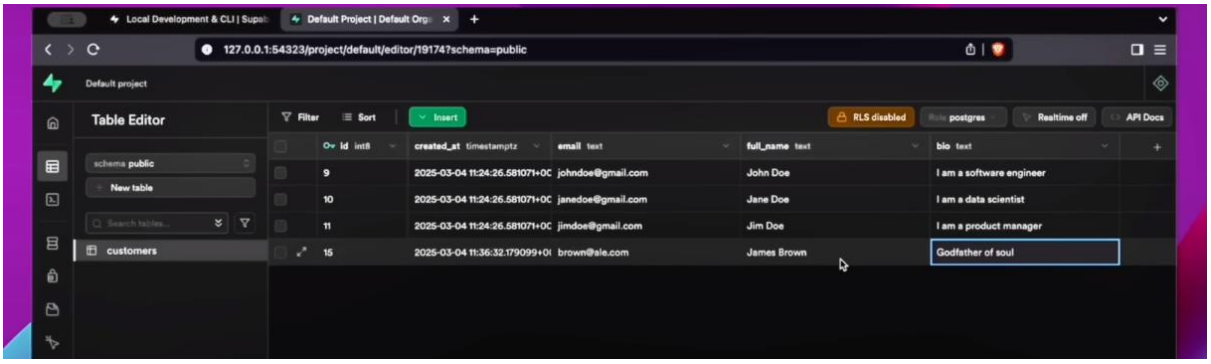
```python
async def get_customer_by_email(ctx: RunContext[Client], email: str):
        ctx.deps.table("customers")
        .select("*")
        .eq("email", email)
        .execute()
    )

    if response.data:
        return response.data[0]
    else:
        raise ValueError(f"No customer found with email: {email}")

# Update tool
@agent.tool(re

# Seed function to create table in Supabase, populate with sample data
def seed_db():
    try:
        response = (
            supabase.table("customers")
            .insert([
                {"email": 'johndoe@gmail.com', "full_name": "John Doe", "bio": "I am a software engineer"},
                {"email": 'janedoe@gmail.com', "full_name": "Jane Doe", "bio": "I am a data scientist"},
                {"email": 'jimdoe@gmail.com', "full_name": "Jim Doe", "bio": "I am a product manager"},
            ])
            .execute()
        )
        return response
    except Exception as exception:
```

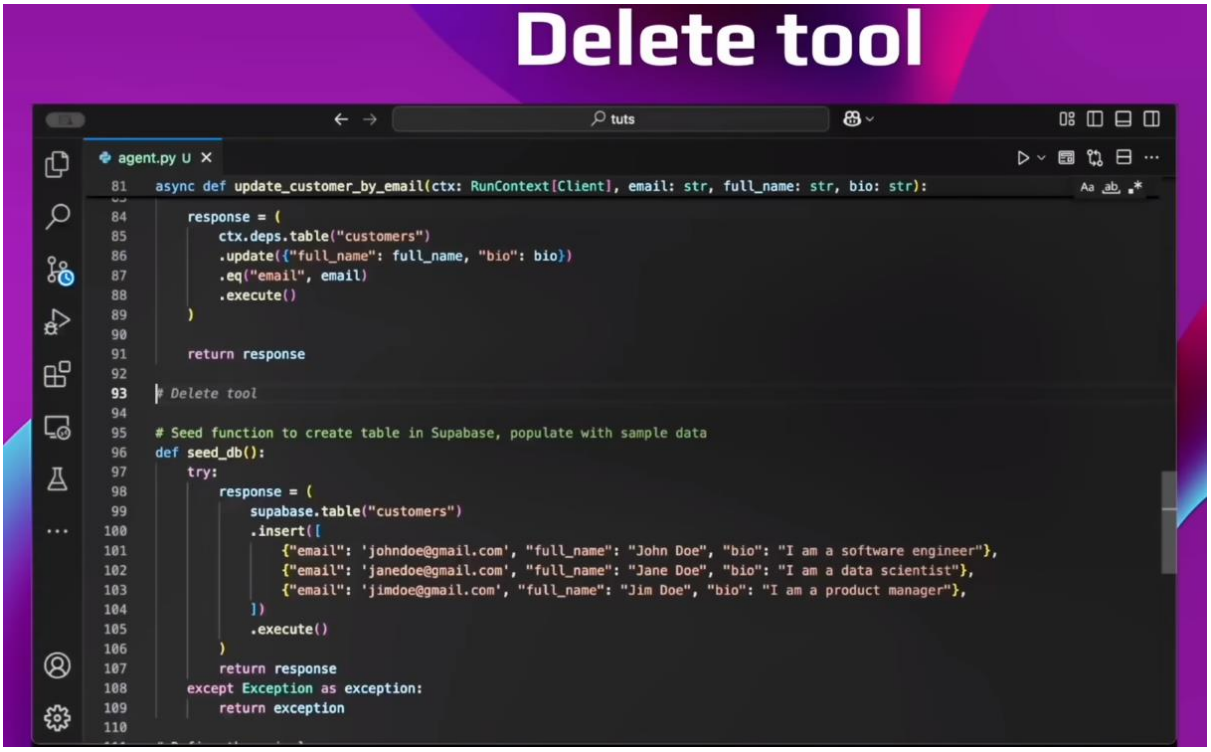


```python
        raise ValueError(f"No customer found with email: {email}")

# Update tool
@agent.tool(retries=3)
async def update_customer_by_email(ctx: RunContext[Client], email: str, full_name: str, bio: str):
    """Update a customer record from their email address."""

    response = (
        ctx.deps.table("customers")
        .update({"full_name": full_name, "bio": bio})
        .eq("email", email)
        .execute()
    )

    return response"""

# Seed function to create table in Supabase, populate with sample data
def seed_db():
    try:
        response = (
            supabase.table("customers")
            .insert([
                {"email": 'johndoe@gmail.com', "full_name": "John Doe", "bio": "I am a software engineer"},
                {"email": 'janedoe@gmail.com', "full_name": "Jane Doe", "bio": "I am a data scientist"},
                {"email": 'jimdoe@gmail.com', "full_name": "Jim Doe", "bio": "I am a product manager"},
            ])
            .execute()
        )
        return response
```

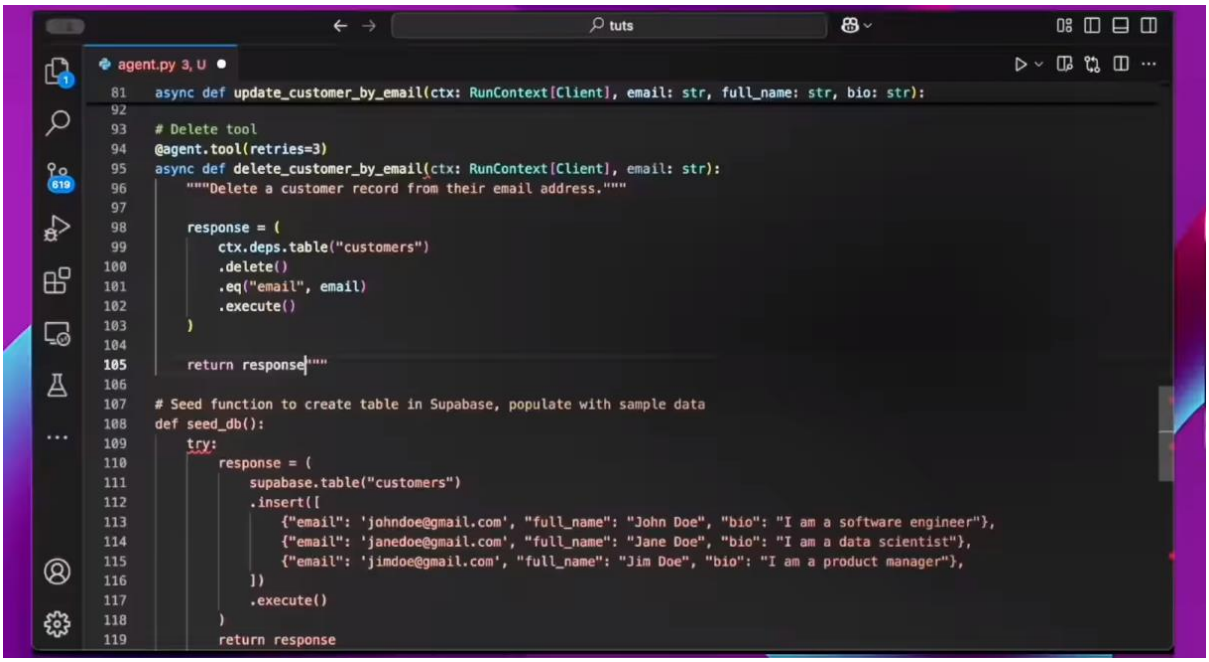


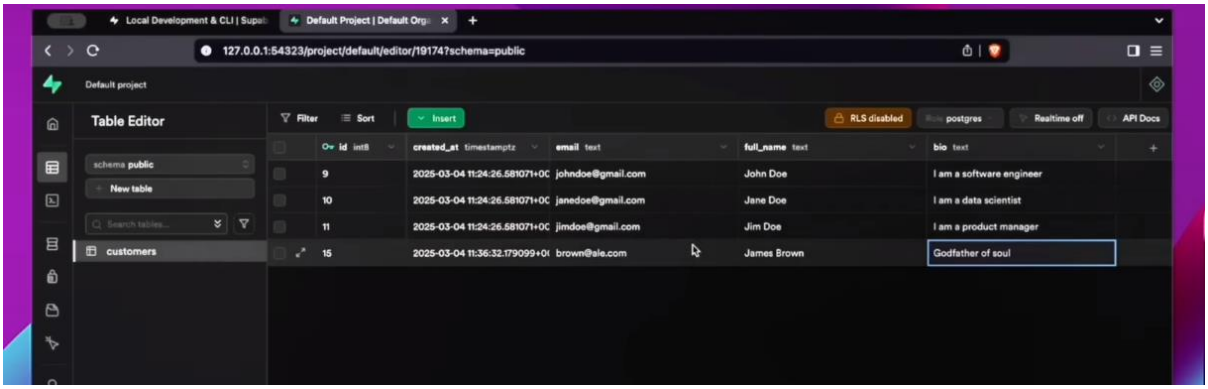


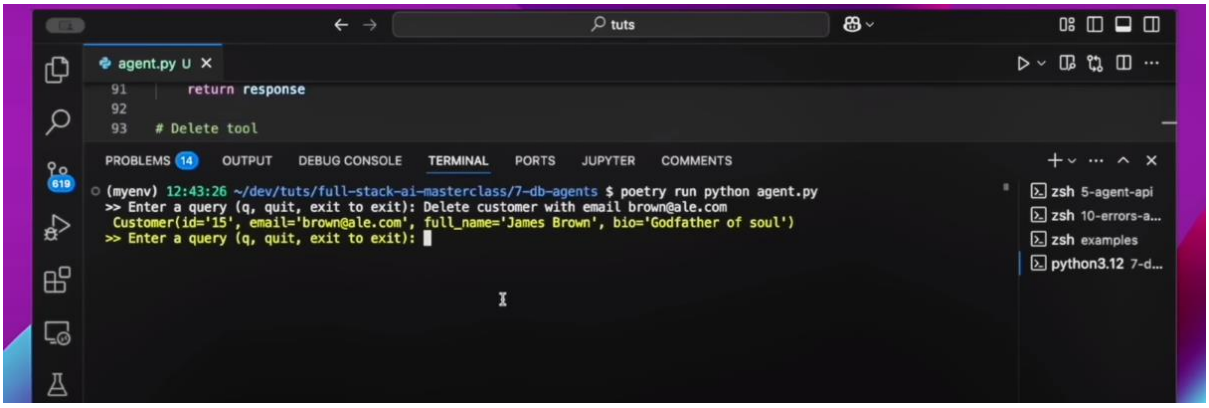


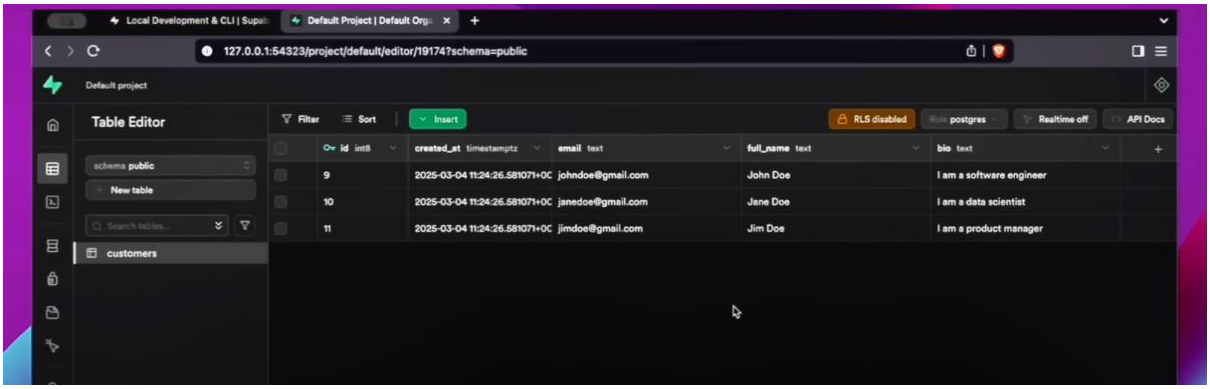


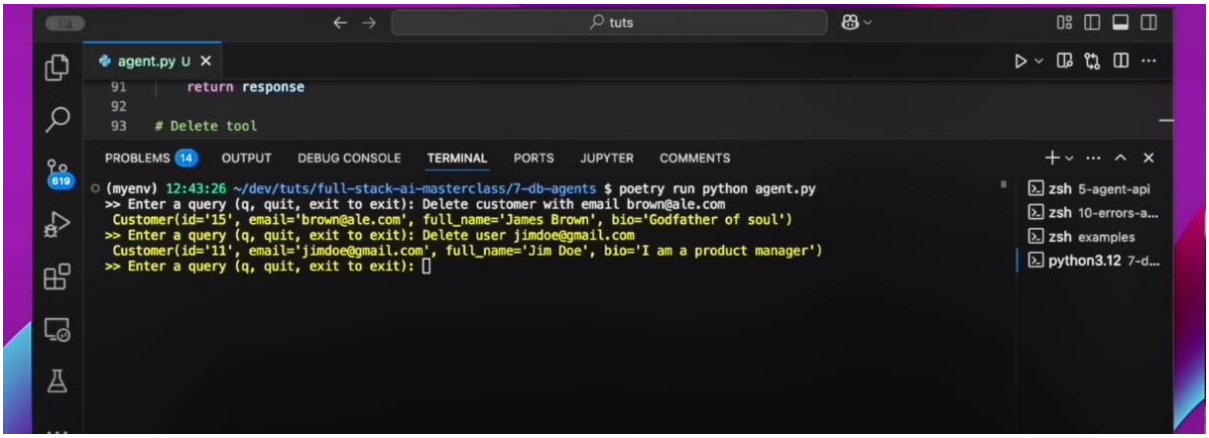
The update works as expected

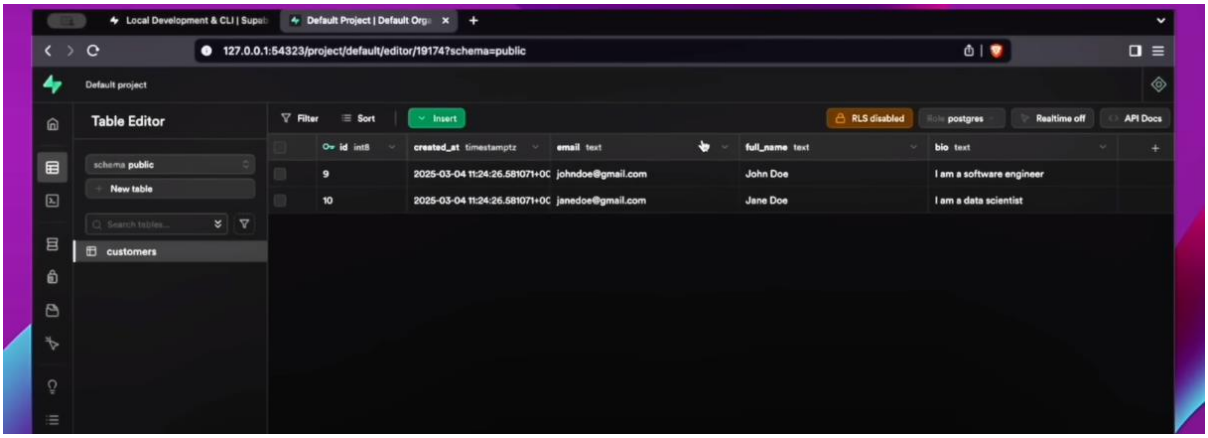

tuts
agent.py 3, U
async def update_customer_by_email(ctx: RunContext[Client], email: str, full_name: str, bio: str):
Delete tool
@agent.tool(retries=3)
async def delete_customer_by_email(ctx: RunContext[Client], email: str):
"""Delete a customer record from their email address."""
response = (
ctx.deps.table("customers")
.delete()
.eq("email", email)
.execute()
)
return response"""
# Seed function to create table in Supabase, populate with sample data
def seed_db():
try:
response = (
supabase.table("customers")
.insert([
{"email": 'johndoe@gmail.com', "full_name": "John Doe", "bio": "I am a software engineer"},
{"email": 'janedoe@gmail.com', "full_name": "Jane Doe", "bio": "I am a data scientist"},
{"email": 'jimdoe@gmail.com', "full_name": "Jim Doe", "bio": "I am a product manager"},
])
.execute()
)
return response


127.0.0.1:54323/project/default/editor/19174?schema=public
Default project
Table Editor
schema public
New table
customers
Filter
Sort
Insert
RLS disabled
postgres
Realtime off
API Docs
id int8
created_at timestamptz
email text
full_name text
bio text
9 2025-03-04 11:24:26.581071+0C johndoe@gmail.com John Doe I am a software engineer
10 2025-03-04 11:24:26.581071+0C janedoe@gmail.com Jane Doe I am a data scientist
11 2025-03-04 11:24:26.581071+0C jimdoe@gmail.com Jim Doe I am a product manager
15 2025-03-04 11:36:32.179099+0(brown@ale.com James Brown Godfather of soul


tuts
agent.py U
return response
Delete tool
PROBLEMS 14
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
JUPYTER
COMMENTS
(myenv) 12:43:26 ~/dev/tuts/full-stack-ai-masterclass/7-db-agents $ poetry run python agent.py
>> Enter a query (q, quit, exit to exit): Delete customer with email brown@ale.com
Customer(id='15', email='brown@ale.com', full_name='James Brown', bio='Godfather of soul')
>> Enter a query (q, quit, exit to exit):
zsh 5-agent-api
zsh 10-errors-a...
zsh examples
python3.12 7-d...


127.0.0.1:54323/project/default/editor/19174?schema=public
Default project
Table Editor
schema public
New table
customers
Filter
Sort
Insert
RLS disabled
postgres
Realtime off
API Docs
id int8
created_at timestamptz
email text
full_name text
bio text
9 2025-03-04 11:24:26.581071+0C johndoe@gmail.com John Doe I am a software engineer
10 2025-03-04 11:24:26.581071+0C janedoe@gmail.com Jane Doe I am a data scientist
11 2025-03-04 11:24:26.581071+0C jimdoe@gmail.com Jim Doe I am a product manager





This concludes our 4 DB CRUD tools for the agent